



**Институт
интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Направление подготовки 09.03.04 Программная инженерия

Пояснительная записка

к учебно-исследовательской работе студента на тему:

**Разработка алгоритмов квантизации векторных представлений для
графовых индексов приближенного поиска ближайших соседей**

Группа	_____ Б23-544 _____		
Студент	_____ (подпись) _____	_____ Минасов М. Р. _____ (ФИО)	
Руководитель	_____ (подпись) _____	_____ Климов В. В. _____ (ФИО)	
Научный консультант	_____ (подпись) _____	_____ Широкин К. П. _____ (ФИО)	
Оценка руководителя	_____ (0-30 баллов)	Оценка консультанта	_____ (0-30 баллов)
Итоговая оценка	_____ (0-100 баллов)	ECTS	_____

Комиссия

Председатель	_____ (подпись) _____	_____ (ФИО) _____
	_____ (подпись) _____	_____ (ФИО) _____
	_____ (подпись) _____	_____ (ФИО) _____
	_____ (подпись) _____	_____ (ФИО) _____

Москва 2026



**Институт интеллектуальных кибернетических систем
КАФЕДРА КИБЕРНЕТИКИ (№ 22)**

Задание на УИР

Студенту гр. Б23-544 Минасову Марату Рустемовичу

ТЕМА УИР

**Разработка алгоритмов квантизации векторных представлений
для графовых методов приближенного поиска ближайших
соседей**

ЗАДАНИЕ

№ п/п	Содержание работы	Форма отчетности	Срок исполнения	Отметка о выполнении Дата, подпись
1.	Аналитическая часть			
1.1.	Изучение постановки задачи поиска ближайших соседей и приближенного поиска ближайших соседей для векторных представлений	Подраздел ПЗ, список литературы		
1.2.	Анализ графовых методов приближенного поиска ближайших соседей применительно к большим наборам векторных представлений	Подраздел ПЗ		
1.3.	Изучение алгоритма HNSW как основного графового метода, используемого в работе. Анализ его параметров, порядка построения и поиска	Подраздел ПЗ		
1.4.	<i>Оформление расширенного содержания пояснительной записки (РСПЗ)</i>	Текст РСПЗ		
2.	Теоретическая часть			
2.1.	Изучение и сравнительный анализ методов квантизации векторных представлений	Подраздел ПЗ, сравнительная таблица методов		
2.2.	Разработка и адаптация алгоритмов квантизации для вычисления приближенной близости между запросом и документными векторами	Подраздел ПЗ, описание алгоритмов		
2.3.	Оценка ожидаемого влияния квантизации на качество поиска, скорость обработки запросов и объем памяти индекса	Подраздел ПЗ, таблица оценок		
3.	Инженерная часть <i>Инженерная часть отсутствует т.к. работа связана с программной реализацией и анализом, без проектирования физических систем</i>			

4.	Технологическая и практическая часть			
4.1.	Реализация базовых методов квантизации векторных представлений	Исходный код		
4.2.	Реализация методов квантизации с кодовыми книгами и низкобитных методов квантизации	Исходный код		
4.3.	Интеграция реализованных методов квантизации с графовым индексом приближенного поиска ближайших соседей	Исходный код		
4.4.	Реализация средств для измерения полноты поиска, скорости обработки запросов и размера индекса	Исполняемые файлы, исходные тексты тестов и тестовых примеров		
4.5.	Подготовка наборов данных для экспериментального сравнения методов квантизации.	Подраздел ПЗ, таблица характеристик наборов данных		
4.6.	Проведение экспериментов с различными методами квантизации и параметрами графового поиска	Журналы экспериментов, таблицы результатов		
4.7.	Сравнение и анализ методов по полноте поиска, числу обработанных запросов в секунду и объему памяти индекса	Сравнительный анализ, таблицы замеренных результатов		
5.	<i>Оформление пояснительной записки (ПЗ) и иллюстративного материала для доклада.</i>	Текст ПЗ, презентация		

ЛИТЕРАТУРА

1. *Malkov Y. A., Yashunin D. A.* Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Т. 42, № 4. — С. 824—836. — DOI: 10.1109/TPAMI.2018.2889473.
2. *Jégou H., Douze M., Schmid C.* Product Quantization for Nearest Neighbor Search // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2011. — Т. 33, № 1. — С. 117—128. — DOI: 10.1109/TPAMI.2010.57.
3. Optimized Product Quantization for Approximate Nearest Neighbor Search / Т. Ge [и др.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — IEEE Computer Society, 2013. — С. 2946—2953. — DOI: 10.1109/CVPR.2013.379.
4. *Gao J., Long C.* RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search // Proceedings of the ACM on Management of Data. — 2024. — Т. 2, № 3. — 167:1—167:27. — DOI: 10.1145/3654970.

5. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search / J. Gao [и др.] // Proceedings of the ACM on Management of Data. — 2025. — Т. 3, № 3. — 202:1—202:26. — DOI: 10.1145/3725413.
6. TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate / A. Zandieh [и др.] // Computing Research Repository. — 2025. — Т. abs/2504.19874. — DOI: 10.48550/arXiv.2504.19874. — URL: <https://arxiv.org/abs/2504.19874>; arXiv:2504.19874.
7. Aumüller M., Bernhardsson E., Faithfull A. J. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms // Information Systems. — 2020. — Т. 87. — С. 101374. — DOI: 10.1016/j.is.2019.02.006.

Дата выдачи задания:

12.02.2026

Руководитель

Студент

Климов В. В.

Минасов М. Р.

Реферат

Общий объем основного текста, без учета приложений --- 42 страниц, с учетом приложений --- 72. Количество использованных источников --- 22. Количество приложений --- 1.

ПРИБЛИЖЕННЫЙ ПОИСК БЛИЖАЙШИХ СОСЕДЕЙ, ГРАФОВЫЙ ПОИСК, HNSW, КВАНТИЗАЦИЯ, ВЕКТОРНЫЕ ПРЕДСТАВЛЕНИЯ, PRODUCT QUANTIZATION, RABITQ, TURBOQUANT, ПОЛНОТА ПОИСКА, СЖАТИЕ ВЕКТОРОВ.

Целью данной работы является разработка и исследование алгоритмов квантизации векторных представлений для графовых методов приближенного поиска ближайших соседей. В работе рассматривается применение квантизации в составе графового индекса HNSW. Основное внимание уделяется влиянию сжатия векторных представлений на полноту поиска, скорость обработки запросов и объем памяти индекса.

В первой главе проводится обзор и анализ задачи поиска ближайших соседей и приближенного поиска ближайших соседей для векторных представлений. Рассматриваются основные показатели качества и производительности поиска, анализируется графовый подход к приближенному поиску, а также подробно описывается алгоритм HNSW как основной графовый метод, используемый в работе.

Во второй главе описываются методы квантизации векторных представлений, используемые в дальнейшем исследовании. Рассматриваются скалярная квантизация, бинарная квантизация, квантизация с кодовыми книгами, RaBitQ и TurboQuant. Для каждого метода описываются способ кодирования документных векторов, подготовка запроса и вычисление приближенной близости в графовом поиске.

В третьей главе приводится описание программной реализации реализованных методов квантизации и их интеграции с графовым индексом HNSW. Также описываются средства измерения полноты поиска, числа обработанных запросов в секунду и размера индекса. Приводится экспериментальное сравнение методов на наборах данных различной размерности и анализируется влияние квантизации на качество, скорость и потребление памяти.

В приложении А приведены основные фрагменты программной реализации алгоритмов квантизации, их подключения к графовому индексу и запуска экспериментальных измерений.

Содержание

Введение	5
1 Анализ графовых методов приближённого поиска и способов квантизации векторных представлений	7
1.1 Постановка задачи поиска ближайших соседей и приближённого поиска ближайших соседей для векторных представлений	7
1.2 Анализ графовых методов приближённого поиска ближайших соседей применительно к большим наборам векторных представлений	10
1.3 Изучение алгоритма HNSW как основного графового метода. Анализ его параметров, порядка построения и поиска	12
1.3.1 Построение графа HNSW	13
1.3.2 Поиск в HNSW	14
1.3.3 Основные параметры HNSW	14
1.3.4 Вычислительные затраты и потребление памяти	15
1.3.5 Связь HNSW с задачей квантизации	16
1.4 Выводы	17
1.5 Цели и задачи УИР	18
2 Разработка алгоритмов квантизации векторных представлений	20
2.1 Сравнительный анализ методов квантизации векторных представлений	20
2.1.1 Базовые методы квантизации	21
2.1.2 Квантизация с кодовыми книгами	22
2.1.3 Низкобитная квантизация после случайного ортогонального преобразования	24
2.2 Адаптация алгоритмов квантизации для вычисления приближенной близости в графовом поиске	25
2.3 Оценка ожидаемого влияния квантизации на качество поиска, скорость обработки запросов и объем памяти индекса	27
2.4 Выводы	28

3	Программная реализация и экспериментальное исследование алгоритмов кван-	30
	тизации для графового приближенного поиска	
3.1	Общая организация программной реализации	30
3.2	Реализация базовых методов квантизации векторных представлений	31
3.2.1	Скалярная квантизация	32
3.2.2	Бинарная квантизация	32
3.3	Реализация квантизации с кодовыми книгами и низкобитных методов	32
3.3.1	Квантизация с кодовыми книгами	33
3.3.2	RaBitQ и TurboQuant	33
3.4	Интеграция реализованных методов с графовым индексом HNSW	34
3.5	Реализация средств измерения	35
3.6	Подготовка наборов данных для экспериментального сравнения	35
3.7	Проведение экспериментов	36
3.8	Сравнение и анализ результатов	37
3.8.1	Набор данных GloVe-100	37
3.8.2	Набор данных Deep Image-96	37
3.8.3	Набор данных DBpedia-1536	38
3.8.4	Обобщение результатов	38
3.9	Выводы	39
	Заключение	41
	Приложения	46
A	Листинг программной реализации алгоритмов квантизации	46

Введение

Векторные представления объектов широко используются в задачах информационного поиска, рекомендаций, поиска похожих изображений и обработки естественного языка. В таких системах документы, изображения или запросы представляются числовыми векторами, а поиск сводится к нахождению ближайших векторов в пространстве большой размерности. Интерес к данной области связан с ростом объёмов данных и распространением моделей машинного обучения, порождающих высокоразмерные представления. При миллионах и миллиардах векторов точный поиск ближайших соседей требует сравнения запроса со всеми объектами базы, что становится слишком затратным по времени и памяти.

Задача приближенного поиска ближайших соседей развивалась в нескольких направлениях. В 1998 году Индик и Мотвани предложили подход на основе хэширования с сохранением близости [1]. В 2002 году Чарикар рассмотрел методы оценки близости на основе случайных гиперплоскостей [2]. В 2011 году Жегу, Дуз и Шмид предложили Product Quantization для сжатия векторных представлений при поиске ближайших соседей [3]. В 2020 году Малков и Яшунин описали алгоритм HNSW, ставший одним из наиболее распространенных графовых методов приближенного поиска [4]. В последние годы появились низкобитные методы квантизации высокоразмерных векторов, включая RaBitQ [5] и TurboQuant [6].

Несмотря на развитие методов приближенного поиска, остается важная практическая проблема: графовые индексы требуют значительного объема памяти, так как необходимо хранить не только структуру графа, но и сами векторные представления. Квантизация позволяет уменьшить размер векторов, однако вносит ошибку в вычисление близости между запросом и документным вектором. Для графового поиска эта ошибка особенно существенна, поскольку она влияет не только на итоговое упорядочивание найденных объектов, но и на путь обхода графа.

Целью данной работы является разработка и исследование алгоритмов квантизации векторных представлений для графовых методов приближенного поиска ближайших соседей. В качестве основного графового метода рассматривается HNSW. В работе изучаются и реализуются скалярная квантизация, бинарная квантизация, Product Quantization, RaBitQ и TurboQuant. Сравнение методов проводится по полноте поиска, числу обработанных запросов в секунду и объему памяти индекса.

Новизна работы состоит в приведении различных методов квантизации к единой схеме

использования внутри графового поиска. Для каждого метода рассматриваются одни и те же этапы: кодирование документных векторов, подготовка запроса и вычисление приближенной близости. Это позволяет сравнивать методы при одинаковых параметрах HNSW и оценивать именно влияние способа квантизации на работу графового индекса.

Оригинальная суть исследования заключается в рассмотрении квантизации не только как способа уменьшения размера векторов, но и как фактора, изменяющего поведение графового поиска. В работе анализируется, как приближенное вычисление близости влияет на полноту поиска, скорость обработки запросов и размер индекса.

В первой главе рассматривается постановка задачи поиска ближайших соседей и приближенного поиска ближайших соседей для векторных представлений, а также анализируется графовый подход к поиску и алгоритм HNSW.

Во второй главе описываются методы квантизации, используемые в работе: скалярная и бинарная квантизация, Product Quantization, RaBitQ и TurboQuant. Также рассматривается их адаптация для вычисления приближенной близости в графовом поиске.

В третьей главе приводится программная реализация методов квантизации, их интеграция с HNSW и экспериментальное сравнение на наборах данных различной размерности.

1. Анализ графовых методов приближённого поиска и способов квантизации векторных представлений

Аннотация. В данной обзорно-аналитической главе рассматриваются постановка задачи поиска ближайших соседей для векторных представлений, особенности приближённого поиска в пространствах высокой размерности, а также графовые методы приближённого поиска. Основное внимание уделяется алгоритму HNSW как наиболее распространённому графовому методу, применимому к большим наборам векторных представлений. Отдельно анализируются параметры алгоритма, порядок построения графа, процедура поиска и причины, по которым квантизация векторных представлений является важным направлением оптимизации графового поиска.

В данной главе рассматривается проблематика приближённого поиска ближайших соседей для векторных представлений. Такая задача возникает в системах семантического поиска, поиска похожих изображений, рекомендательных системах и других приложениях, где объекты представлены в виде числовых векторов фиксированной размерности. При больших объёмах данных точный перебор всех векторов становится вычислительно неприемлемым, поэтому используются приближённые методы поиска.

Цель аналитической части состоит в том, чтобы определить математическую постановку задачи, рассмотреть основные свойства графовых методов приближённого поиска и обосновать выбор алгоритма HNSW как основы для дальнейшей разработки и исследования алгоритмов квантизации векторных представлений.

1.1 Постановка задачи поиска ближайших соседей и приближённого поиска ближайших соседей для векторных представлений

Пусть задано множество векторных представлений объектов

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R}^d,$$

где n --- число объектов в базе, а d --- размерность векторного представления. Также задан запросный вектор

$$q \in \mathbb{R}^d.$$

Задача поиска ближайших соседей состоит в нахождении объектов из множества X , наиболее близких к запросу q в смысле выбранной функции расстояния или меры близости.

В общем виде вводится функция расстояния

$$\rho : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}.$$

Тогда задача поиска k ближайших соседей заключается в нахождении множества

$$N_k(q) \subset X, \quad |N_k(q)| = k,$$

такого, что для любых $x \in N_k(q)$ и $y \in X \setminus N_k(q)$ выполняется условие

$$\rho(q, x) \leq \rho(q, y).$$

Иными словами, множество $N_k(q)$ содержит k объектов базы, имеющих наименьшее расстояние до запроса.

На практике для векторных представлений наиболее часто используются евклидово расстояние и скалярное произведение. Евклидово расстояние задаётся формулой

$$\rho_2(q, x) = \|q - x\|_2.$$

Скалярное произведение определяется как

$$s(q, x) = \langle q, x \rangle.$$

Если все векторы предварительно нормированы, то есть

$$\|q\|_2 = 1, \quad \|x\|_2 = 1,$$

то евклидово расстояние и скалярное произведение задают один и тот же порядок соседей. Действительно,

$$\|q - x\|_2^2 = \|q\|_2^2 + \|x\|_2^2 - 2\langle q, x \rangle = 2 - 2\langle q, x \rangle.$$

Следовательно, минимизация евклидова расстояния эквивалентна максимизации скалярного произведения. Это свойство важно для дальнейшего исследования, поскольку многие методы квантизации фактически строят приближённую оценку скалярного произведения между

запросом и сжатым представлением объекта.

Наиболее простой способ точного решения задачи поиска ближайших соседей состоит в полном переборе всех векторов базы. Для каждого запроса вычисляется расстояние до каждого объекта x_i , после чего выбираются k объектов с наименьшими расстояниями. Вычислительная сложность такого подхода составляет

$$O(n \cdot d)$$

на один запрос, так как для каждого из n объектов необходимо обработать d координат.

При малом числе объектов полный перебор может быть допустимым и даже предпочтительным из-за простоты реализации. Однако при миллионах и миллиардах векторов такой подход становится слишком дорогим. Например, если база содержит 10^9 векторов размерности $d = 1000$, то один запрос требует порядка 10^{12} элементарных операций только для вычисления расстояний. Для интерактивных поисковых систем, где время ответа ограничено десятками или сотнями миллисекунд, такой объём вычислений неприемлем.

Для уменьшения времени обработки запроса используются методы приближённого поиска ближайших соседей. В теоретической постановке приближённый поиск часто формулируется через допустимую относительную ошибку. Для случая поиска одного ближайшего соседа алгоритм должен вернуть такой объект $\hat{x}$, что

$$\rho(q, \hat{x}) \leq (1 + \varepsilon)\rho(q, x^*),$$

где x^* --- истинный ближайший сосед запроса q , а $\varepsilon > 0$ --- допустимая погрешность. Подобная постановка рассматривается в классических работах по приближённому поиску ближайших соседей [1, 7].

Однако в прикладных системах чаще используется другая оценка качества: доля истинных ближайших соседей, найденных приближённым алгоритмом. Пусть Q --- множество запросов, $N_k(q)$ --- множество точных k ближайших соседей запроса q , а $\hat{N}_k(q)$ --- множество k объектов, возвращённых приближённым алгоритмом. Тогда полнота поиска для первых k результатов определяется как

$$R_k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|N_k(q) \cap \hat{N}_k(q)|}{k}.$$

Значение $R_k = 1$ означает, что алгоритм для всех запросов вернул ровно те же k объектов,

что и точный поиск. Значение $R_k < 1$ означает, что часть истинных ближайших соседей была потеряна.

Для данной работы именно полнота поиска R_k является основной метрикой качества, так как она прямо показывает, насколько сильно приближённый метод отличается от точного поиска. При этом качество необходимо рассматривать совместно с вычислительными и системными показателями:

- временем обработки одного запроса;
- числом запросов, обрабатываемых за единицу времени;
- объёмом памяти, необходимым для хранения индекса;
- временем построения индекса.

В контексте данной работы особое значение имеет объём памяти. Графовые методы приближённого поиска, как правило, хранят не только сами векторы, но и связи между ними. Если векторы хранятся в формате чисел с плавающей точкой одинарной точности, то только их хранение требует

4nd

байт памяти. При больших n и d этот объём становится одним из основных ограничений. Поэтому возникает задача уменьшения объёма векторных представлений при сохранении приемлемого качества поиска. Именно эту задачу решают методы квантизации, которые рассматриваются и разрабатываются в последующих главах.

1.2 Анализ графовых методов приближённого поиска ближайших соседей применительно к большим наборам векторных представлений

Графовые методы приближённого поиска основаны на представлении набора векторных объектов в виде графа близости. В таком графе вершины соответствуют документным векторам, а рёбра соединяют объекты, близкие друг к другу в исходном векторном пространстве. Формально граф можно записать как

$$G = (V, E),$$

где V — множество вершин, соответствующих векторам $x_1, x_2, \dots, x_n$, а E — множество рёбер между ними.

Основная идея графового поиска состоит в том, что для обработки запроса не требуется сравнивать его со всеми объектами базы. Алгоритм начинает обход с одной или нескольких входных вершин, после чего последовательно переходит к соседним вершинам, которые оказываются ближе к запросу. При удачной структуре графа такой обход приводит в область пространства, где расположены ближайшие к запросу объекты.

Простейший вариант такого поиска является жадным. Пусть текущей является вершина v . Алгоритм рассматривает соседей этой вершины и переходит в того соседа u , для которого расстояние до запроса минимально:

$$u = \arg \min_{w \in N(v)} \rho(q, x_w),$$

где $N(v)$ — множество соседей вершины v . Если среди соседей нет вершины, которая ближе к запросу, чем текущая вершина, обход останавливается.

Такой подход позволяет существенно уменьшить число проверяемых объектов по сравнению с полным перебором. Однако у жадного поиска есть важное ограничение: он может остановиться в локальном минимуме. Локальный минимум возникает в том случае, если текущая вершина ближе к запросу, чем все её соседи, но в другой части графа существуют вершины, расположенные к запросу значительно ближе.

Поэтому при построении графового индекса необходимо обеспечить хорошие навигационные свойства графа. С одной стороны, вершины должны иметь рёбра к близким объектам, чтобы можно было уточнять результат в локальной области. С другой стороны, в графе должны существовать переходы между разными областями пространства, иначе поиск может застревать в локальных минимумах.

При построении графа возникает компромисс между качеством поиска и размером индекса:

- если хранить слишком мало рёбер, граф может оказаться плохо связным, а полнота поиска снизится;
- если хранить слишком много рёбер, увеличиваются объём памяти, время построения индекса и время обработки запроса;
- если связи в графе недостаточно разнообразны, поиск может плохо переходить между разными областями пространства.

На практике одним из наиболее распространённых графовых методов является HNSW — иерархический навигируемый граф малого мира [4]. Его популярность связана с тем, что он

обеспечивает высокую полноту поиска, имеет понятные параметры настройки и показывает хорошие результаты на векторных представлениях большой размерности.

Помимо HNSW, в области приближенного поиска рассматривались пространственные индексы, методы хэширования и автоматизированные библиотеки выбора индекса [8—10].

В данной работе дальнейший анализ ограничивается алгоритмом HNSW. Такой выбор обусловлен тем, что целью работы является не сравнение различных графовых индексов между собой, а исследование влияния квантизации векторных представлений на графовый приближённый поиск. HNSW выбран как основной графовый метод, на примере которого можно изучить, как приближённое вычисление близости влияет на обход графа, полноту поиска, скорость обработки запросов и объём памяти индекса.

1.3 Изучение алгоритма HNSW как основного графового метода. Анализ его параметров, порядка построения и поиска

Алгоритм HNSW был предложен как развитие идей графов малого мира и списков с пропусками [11—13]. Основная идея состоит в построении нескольких уровней графа. Верхние уровни содержат меньше вершин и более длинные переходы, позволяющие быстро приблизиться к области запроса. Нижний уровень содержит все вершины и используется для более точного поиска ближайших соседей.

Индекс HNSW можно представить как набор графов

$$G_0, G_1, \dots, G_L,$$

где G_0 --- нижний уровень, содержащий все вершины, а G_L --- самый верхний уровень. Каждая вершина присутствует на нижнем уровне и, с некоторой вероятностью, на нескольких более высоких уровнях. Максимальный уровень вершины выбирается случайно. В классическом описании алгоритма уровень вершины определяется выражением

$$l(v) = \lfloor -\ln u \cdot m_L \rfloor, \quad u \sim U(0, 1),$$

где u --- случайная величина, равномерно распределённая на интервале $(0, 1)$, а m_L --- параметр, управляющий распределением числа вершин по уровням. Такое правило приводит к тому, что число вершин на уровнях убывает примерно экспоненциально.

Эта структура напоминает списки с пропусками: верхние уровни используются для быстрой навигации, а нижний уровень --- для уточнения результата. Если бы алгоритм использовал только нижний граф, то поиск мог бы требовать большого числа локальных переходов.

Если же добавить верхние уровни, то поиск сначала быстро перемещается на большие расстояния, а затем постепенно уточняет положение на нижних уровнях.

1.3.1 Построение графа HNSW

Построение HNSW выполняется последовательно: векторы добавляются в индекс один за другим. При добавлении нового вектора x сначала случайно выбирается максимальный уровень, на котором будет присутствовать соответствующая вершина. Далее алгоритм ищет подходящие соседние вершины на каждом уровне и добавляет рёбра между новой вершиной и уже существующими вершинами.

Процедура вставки включает следующие основные этапы.

1. Выбирается максимальный уровень $l(x)$ для новой вершины.
2. Начиная с верхнего уровня текущего графа, выполняется жадный спуск к области, в которой расположен добавляемый вектор.
3. На каждом уровне, где должна присутствовать новая вершина, выполняется поиск нескольких ближайших кандидатов.
4. Из найденных кандидатов выбирается ограниченное число соседей.
5. Добавляются рёбра от новой вершины к выбранным соседям, а также обратные рёбра от соседей к новой вершине.
6. Если число рёбер у какой-либо вершины становится слишком большим, список её соседей сокращается.

Важной частью построения является выбор соседей. Наивный способ состоял бы в том, чтобы соединять вершину только с ближайшими кандидатами. Однако такой подход может приводить к появлению слишком локальных связей: все рёбра будут направлены в одну плотную область пространства, а переходы в другие направления окажутся затруднены.

Поэтому в HNSW применяется эвристика выбора разнообразных соседей. Пусть R --- уже выбранный набор соседей для новой вершины, а e --- очередной кандидат. Кандидат добавляется в R , если он ближе к новой вершине, чем к любому уже выбранному соседу:

$$\rho(e, x) < \rho(e, r), \quad \forall r \in R.$$

Смысл этого условия состоит в том, что новый сосед должен давать новое направление для обхода графа, а не дублировать уже выбранную связь. Благодаря этому граф становится более пригодным для навигации.

1.3.2 Поиск в HNSW

Поиск в HNSW также выполняется сверху вниз. Сначала алгоритм начинает работу с некоторой входной вершины на самом верхнем уровне. На каждом верхнем уровне выполняется жадный поиск: если среди соседей текущей вершины есть вершина, более близкая к запросу, алгоритм переходит в неё. Когда улучшение становится невозможным, найденная вершина используется как входная точка для следующего, более низкого уровня.

На нижнем уровне G_0 применяется более широкий поиск. Вместо того чтобы хранить только одну текущую вершину, алгоритм поддерживает множество кандидатов и множество лучших найденных вершин. Это позволяет исследовать не одно направление, а несколько возможных путей в графе.

Процедура поиска на нижнем уровне состоит из следующих шагов.

1. Создаётся очередь кандидатов, упорядоченная по расстоянию до запроса.
2. Создаётся множество текущих лучших найденных вершин.
3. Из очереди извлекается ближайший ещё не обработанный кандидат.
4. Рассматриваются его соседи. Если сосед достаточно близок к запросу, он добавляется в очередь кандидатов и в множество лучших вершин.
5. Поиск завершается, когда ближайший необработанный кандидат уже не может улучшить текущее множество найденных вершин.

После завершения поиска из найденного множества выбираются k ближайших вершин. Они и образуют ответ алгоритма.

1.3.3 Основные параметры HNSW

Качество и скорость работы HNSW определяются несколькими параметрами. Наиболее важные из них приведены в таблице 1.1.

Параметр M определяет степень связности графа. Если M слишком мал, то граф может оказаться недостаточно хорошо навигируемым, и поиск будет чаще останавливаться в ло-

Таблица 1.1 – Основные параметры алгоритма HNSW

Параметр	Назначение	Влияние на работу алгоритма
M	Максимальное число соседей вершины на уровне графа	При увеличении M обычно растёт полнота поиска, но также растут объём памяти и время построения индекса
$efConstruction$	Размер множества кандидатов, рассматриваемых при построении графа	Большие значения улучшают качество построенного графа, но увеличивают время построения индекса
$efSearch$	Размер множества кандидатов, рассматриваемых во время поиска	Позволяет управлять соотношением между полнотой поиска и временем обработки запроса без перестроения индекса

кальных областях. Если M слишком велик, то каждый шаг поиска требует просмотра большого числа соседей, а сам граф занимает больше памяти.

Параметр $efConstruction$ влияет только на построение индекса. Чем больше кандидатов рассматривается при вставке каждой вершины, тем выше вероятность найти качественных соседей. Однако увеличение этого параметра приводит к росту времени построения.

Параметр $efSearch$ используется во время обработки запроса. Он является одним из основных инструментов управления качеством поиска. При малом $efSearch$ алгоритм работает быстрее, но может пропустить часть истинных ближайших соседей. При большом $efSearch$ алгоритм рассматривает больше вершин, что обычно повышает полноту поиска, но увеличивает время обработки запроса.

1.3.4 Вычислительные затраты и потребление памяти

Время поиска в HNSW зависит от числа посещённых вершин и стоимости вычисления расстояния от запроса до каждой из них. Если расстояние вычисляется по исходным векторам размерности d , то одна такая операция имеет сложность $O(d)$. Поэтому даже при малом числе посещённых вершин высокая размерность векторных представлений может существенно влиять на время обработки запроса.

Потребление памяти в HNSW складывается из двух основных частей:

- памяти для хранения векторных представлений;
- памяти для хранения рёбер графа и служебных структур.

Если каждый вектор хранится в формате числа с плавающей точкой одинарной точности, то объём памяти для хранения векторов равен

$$S_{\text{vec}} = 4nd.$$

Объём памяти для хранения графа можно оценить как

$$S_{\text{graph}} \approx nMs_{\text{id}},$$

где s_{id} --- размер идентификатора вершины в байтах. В реальных реализациях эта оценка может отличаться из-за хранения дополнительных данных, разных ограничений на число рёбер на нижнем и верхних уровнях, а также служебных структур. Тем не менее эта формула показывает основной характер зависимости: память на граф растёт линейно по числу вершин и по параметру M , а память на векторы дополнительно растёт линейно по размерности d .

Из этих оценок следует важный вывод. При высокой размерности векторов память на хранение самих векторных представлений может существенно превосходить память на хранение рёбер графа. Например, если $d = 1536$, то один вектор в формате чисел с плавающей точкой одинарной точности занимает

$$1536 \cdot 4 = 6144$$

байта. При миллионах объектов это приводит к нескольким гигабайтам памяти только на векторные представления. Поэтому уменьшение размера векторов является важным способом оптимизации графового поиска.

1.3.5 Связь HNSW с задачей квантизации

В рамках данной работы квантизация рассматривается как способ заменить исходное векторное представление x его сжатым кодом $c(x)$. Вместо точного расстояния $\rho(q, x)$ или точного скалярного произведения $\langle q, x \rangle$ используется приближённая оценка

$$\widehat{\rho}(q, c(x)) \quad \text{или} \quad \widehat{s}(q, c(x)).$$

Такой подход позволяет уменьшить объём памяти, необходимый для хранения векторных представлений, а также потенциально ускорить вычисление близости между запросом и вершинами графа.

Однако применение квантизации в HNSW имеет важную особенность. В графовом поиске приближённые расстояния используются не только для окончательного упорядочивания найденных объектов, но и для выбора пути обхода графа. Если ошибка квантизации изменит порядок расстояний между кандидатами, алгоритм может перейти не в ту вершину и пропустить область, где находятся истинные ближайшие соседи. Поэтому качество квантизации влияет не только на итоговое сравнение кандидатов, но и на сам процесс навигации по графу.

С другой стороны, уменьшение размера векторов может положительно сказаться на скорости поиска. Сжатые представления требуют меньшего объёма чтения из памяти. Кроме того, некоторые способы квантизации позволяют вычислять оценки близости с помощью более простых операций, чем вычисление скалярного произведения по исходным вещественным координатам.

Таким образом, при использовании квантизации в HNSW возникает необходимость исследовать компромисс между тремя показателями:

- полнотой поиска;
- скоростью обработки запросов;
- объёмом памяти индекса.

Именно этот компромисс является центральным для дальнейшей работы. В последующих главах будут рассмотрены и реализованы алгоритмы квантизации, которые позволяют получать сжатые представления векторов и использовать их при поиске по графовому индексу.

1.4 Выводы

В результате проведённого анализа можно сформулировать следующие выводы.

1. Изучена постановка задачи точного и приближённого поиска ближайших соседей для векторных представлений. Показано, что полный перебор имеет сложность $O(n \cdot d)$ на один запрос и становится непрактичным при больших объёмах данных.
2. Определены основные показатели, по которым далее сравниваются методы квантизации: полнота поиска, скорость обработки запросов и объём памяти индекса.
3. Рассмотрен графовый подход к приближённому поиску ближайших соседей. Показано, что графовый поиск позволяет уменьшить число проверяемых объектов, но его качество зависит от навигационных свойств графа и точности вычисления близости.

4. Изучен алгоритм HNSW, его иерархическая структура, порядок построения и процедура поиска. HNSW выбран как основной графовый метод, используемый в работе.
5. Обоснована необходимость квантизации векторных представлений для графового поиска. При высокой размерности значительная часть памяти расходуется на хранение самих векторов, поэтому их сжатие может заметно уменьшить размер индекса.

1.5 Цели и задачи УИР

Проведённый анализ показывает, что задача приближённого поиска ближайших соседей является одной из ключевых задач при работе с большими наборами векторных представлений. Точный поиск требует вычисления расстояний до всех объектов базы, что приводит к чрезмерным вычислительным затратам при больших объёмах данных. Графовые методы, в частности HNSW, позволяют существенно сократить число проверяемых объектов и обеспечивают высокую полноту поиска. Однако для эффективной работы графового индекса необходимо хранить как рёбра графа, так и сами векторные представления. При высокой размерности именно хранение векторов становится одним из основных источников потребления оперативной памяти.

Целью данной учебно-исследовательской работы является разработка и экспериментальное исследование алгоритмов квантизации векторных представлений для графовых методов приближённого поиска ближайших соседей. Разрабатываемые алгоритмы должны позволять уменьшать объём памяти, требуемый для хранения векторных представлений в графовом индексе, при сохранении приемлемой полноты поиска и скорости обработки запросов.

Объектом исследования являются графовые методы приближённого поиска ближайших соседей для векторных представлений. Предметом исследования являются алгоритмы квантизации векторных представлений и их влияние на качество, скорость и потребление памяти при поиске по графовому индексу.

Для достижения поставленной цели необходимо решить следующие задачи.

1. Изучить постановку задачи поиска ближайших соседей и приближённого поиска ближайших соседей для векторных представлений. Определить метрики, по которым будет выполняться сравнение методов: полнота поиска, скорость обработки запросов и объём памяти индекса.
2. Изучить графовые методы приближённого поиска ближайших соседей и подробно рассмотреть алгоритм HNSW как основной графовый метод, используемый в работе. Вы-

явить, какие операции алгоритма наиболее чувствительны к способу хранения и сравнения векторных представлений.

3. Изучить существующие методы квантизации векторных представлений, применимые к задаче приближённого поиска. Рассмотреть базовые методы квантизации, методы с кодовыми книгами и современные низкобитные методы.
4. Разработать и адаптировать алгоритмы квантизации для использования внутри графового поиска. Для каждого метода необходимо определить способ кодирования вектора, способ подготовки запроса и способ вычисления приближённой меры близости между запросом и сжатым представлением объекта.
5. Реализовать выбранные алгоритмы квантизации и интегрировать их с графовым индексом приближённого поиска ближайших соседей. Реализация должна позволять проводить эксперименты с различными параметрами квантизации и поиска.
6. Провести экспериментальное сравнение реализованных методов на наборах векторных представлений различной размерности. Для каждого метода необходимо измерить полноту поиска, скорость обработки запросов и объём памяти индекса.
7. На основании результатов экспериментов определить, какие методы квантизации дают наиболее выгодное соотношение между качеством поиска, скоростью и потреблением памяти при использовании графового приближённого поиска.

Результатом работы должны стать реализованные алгоритмы квантизации векторных представлений, применимые к графовому приближённому поиску ближайших соседей, а также экспериментальная оценка их эффективности. Особое внимание в работе уделяется тому, как ошибка квантизации влияет на обход графа HNSW и на итоговую полноту поиска.

2. Разработка алгоритмов квантизации векторных представлений

Аннотация. В данной главе рассматриваются методы квантизации векторных представлений, используемые далее в программной реализации и экспериментальном сравнении. Описываются скалярная и бинарная квантизация, квантизация с кодовыми книгами, а также низкобитные методы *RaBitQ* и *TurboQuant*. Для каждого метода рассматривается способ кодирования документных векторов, подготовка запроса и вычисление приближенной близости. Также оценивается ожидаемое влияние квантизации на полноту поиска, скорость обработки запросов и объем памяти графового индекса.

В первой главе было показано, что графовые методы приближенного поиска ближайших соседей позволяют существенно уменьшить число проверяемых объектов по сравнению с полным перебором. Однако при высокой размерности векторных представлений значительная часть памяти индекса расходуется на хранение самих векторов. Кроме того, при обработке запроса требуется многократно вычислять близость между запросом и вершинами графа.

Квантизация решает эту проблему за счет замены исходного вещественного вектора более компактным кодом. Такой код должен занимать меньше памяти, но при этом позволять достаточно точно оценивать близость между запросом и документным вектором. В графовом поиске это особенно важно, поскольку приближенные значения близости используются не только для итогового выбора кандидатов, но и для выбора направления обхода графа.

2.1 Сравнительный анализ методов квантизации векторных представлений

Методы квантизации векторов опираются на классические результаты теории векторного квантования [14—16].

Квантизацией векторного представления называется отображение исходного вектора $x \in \mathbb{R}^d$ в компактный код $c(x)$. В общем виде можно выделить две операции:

$$E : \mathbb{R}^d \rightarrow \mathcal{C},$$

$$D : \mathcal{C} \rightarrow \mathbb{R}^d,$$

где E — кодирование, D — восстановление, а $\mathcal{C}$ — множество допустимых кодов. Код вектора имеет вид

$$c(x) = E(x),$$

а восстановленное приближение вектора задается как

$$\hat{x} = D(c(x)).$$

В классической задаче сжатия качество квантизации часто оценивают через ошибку восстановления:

$$\|x - \hat{x}\|_2^2.$$

Однако в задаче поиска ближайших соседей важнее не само восстановление вектора, а сохранение порядка близости объектов к запросу. Поэтому основное значение имеет ошибка приближенного вычисления близости:

$$|\langle q, x \rangle - \hat{s}(q, c(x))|,$$

где $\hat{s}(q, c(x))$ — приближенная оценка близости между запросом и сжатым представлением документного вектора.

В данной работе рассматриваются следующие методы квантизации:

- скалярная квантизация;
- бинарная квантизация;
- квантизация с кодовыми книгами;
- RaBitQ;
- TurboQuant.

2.1.1 Базовые методы квантизации

Скалярная квантизация применяется независимо к каждой координате вектора. Для координаты x_j выбирается конечное число допустимых уровней, после чего исходное значение заменяется номером ближайшего уровня. Если для координаты известен диапазон $[a_j, b_j]$, а число бит на координату равно r , то число уровней равно

$$L = 2^r.$$

Шаг квантизации:

$$\Delta_j = \frac{b_j - a_j}{L - 1}.$$

Код координаты вычисляется как

$$c_j(x) = \text{clip} \left(\left\lfloor \frac{x_j - a_j}{\Delta_j} \right\rfloor, 0, L - 1 \right).$$

Восстановленное значение координаты:

$$\hat{x}_j = a_j + \Delta_j c_j(x).$$

На практике часто используется восьмибитная скалярная квантизация. Она уменьшает объем хранения координат примерно в четыре раза по сравнению с форматом float32 и обычно сохраняет достаточно высокое качество поиска. Главный недостаток метода состоит в том, что координаты квантуются независимо, поэтому связи между ними не учитываются.

Бинарная квантизация является предельным случаем сильного сжатия. В простейшем варианте каждая координата заменяется одним битом по знаку:

$$b_j(x) = \begin{cases} 1, & x_j \geq 0, \\ 0, & x_j < 0. \end{cases}$$

После этого близость между запросом и документом можно оценивать через расстояние Хэмминга:

$$H(b(q), b(x)) = \sum_{j=1}^d \mathbb{I}[b_j(q) \neq b_j(x)].$$

В вычислительном отношении бинарная квантизация очень выгодна, так как расстояние Хэмминга вычисляется с помощью побитовых операций. При этом один вектор занимает всего d бит. Однако качество такого представления может быть низким, поскольку величины координат полностью теряются. Связь бинарных кодов с угловой близостью также рассматривается в методах случайных гиперплоскостей [2].

2.1.2 Квантизация с кодовыми книгами

Квантизация с кодовыми книгами заменяет вектор или его часть номером ближайшего эталонного вектора. В данной работе этот подход рассматривается на примере метода произведения квантизаторов, известного как Product Quantization [3].

Исходный вектор разбивается на m блоков:

$$x = (x^{(1)}, x^{(2)}, \dots, x^{(m)}) .$$

Для каждого блока обучается отдельная кодовая книга:

$$A_j = \{a_{j,1}, a_{j,2}, \dots, a_{j,K}\}, \quad j = 1, \dots, m.$$

Код блока определяется номером ближайшего элемента кодовой книги:

$$c_j(x) = \arg \min_{t \in \{1, \dots, K\}} \|x^{(j)} - a_{j,t}\|_2^2 .$$

Полный код вектора имеет вид

$$c(x) = (c_1(x), c_2(x), \dots, c_m(x)).$$

Если $K = 256$, то номер элемента кодовой книги хранится в одном байте. Тогда размер кода одного вектора равен m байтам. Меняя число блоков m , можно управлять соотношением между степенью сжатия и точностью.

Для запроса заранее строятся таблицы близости:

$$T_j(t) = \langle q^{(j)}, a_{j,t} \rangle .$$

После этого приближенное скалярное произведение с документным вектором вычисляется как

$$\widehat{s}_{\text{PQ}}(q, c(x)) = \sum_{j=1}^m T_j(c_j(x)).$$

Таким образом, после подготовки таблиц сравнение запроса с одним документом требует только m обращений к таблицам и m сложений.

Преимущество метода состоит в хорошем соотношении качества и объема памяти. Недостаток состоит в необходимости обучать кодовые книги на выборке данных. Кроме того, качество зависит от выбранного разбиения вектора на блоки. Существуют и другие варианты квантизации с кодовыми книгами и ускоренного вычисления близости, развивающие идеи Product Quantization [Guo2020AnisotropicVectorQuantization, 17, 18].

2.1.3 Низкобитная квантизация после случайного ортогонального преобразования

Методы RaBitQ и TurboQuant используют общую идею: перед квантизацией к вектору применяется случайное ортогональное преобразование. Пусть P — ортогональная матрица, то есть

$$P^T P = I.$$

Для документного вектора x вычисляется его норма

$$r_x = \|x\|_2,$$

после чего строится преобразованный нормированный вектор

$$y = \frac{Px}{r_x}.$$

Так как P ортогональна, скалярные произведения сохраняются. Поэтому квантировать можно преобразованный вектор y , а запрос перед сравнением также преобразовывать с помощью P .

Смысл такого преобразования состоит в том, что после случайного поворота координаты вектора становятся более однородными. Для высокоразмерных нормированных векторов отдельные координаты после такого преобразования обычно имеют малую величину и близкое распределение. Это делает возможным низкобитное покоординатное кодирование.

В методе RaBitQ в простейшем варианте хранится знак каждой координаты преобразованного вектора [5]. В многобитном варианте используется равномерная сетка уровней [19]. При обработке запроса вычисляется

$$z = Pq.$$

Приближенное скалярное произведение можно записать в виде

$$\hat{s}_{\text{RaBitQ}}(q, c(x)) = r_x \sum_{i=1}^d z_i \hat{y}_i,$$

где $\hat{y}_i$ — восстановленное значение координаты после квантизации. В практических вариантах метода могут использоваться дополнительные скалярные поправки, уменьшающие ошибку оценки.

TurboQuant использует ту же общую схему со случайным ортогональным преобразованием, но отличается выбором уровней квантизации [6]. В RaBitQ многобитный вариант исполь-

зует равномерную сетку, а TurboQuant использует неравномерные уровни, заранее согласованные с распределением координат после случайного поворота. Так как такие координаты чаще принимают значения около нуля, уровни квантизации располагаются плотнее в этой области. Это позволяет уменьшить ошибку кодирования при том же числе бит на координату.

Важное свойство RaBitQ и TurboQuant состоит в том, что они не требуют обучения кодовых книг на конкретном наборе данных. Их параметры в основном задаются размерностью векторов, числом бит на координату и выбранным ортогональным преобразованием. Поэтому эти методы особенно интересны для высокоразмерных векторных представлений.

Сравнение рассматриваемых методов приведено в таблице 2.1.

Таблица 2.1 – Сравнение методов квантизации векторных представлений

Метод	Основная идея	Преимущества	Ограничения
Скалярная квантизация	Независимое кодирование каждой координаты	Простая реализация, устойчивое качество при 8 битах на координату	Ограниченная степень сжатия, не учитываются связи между координатами
Бинарная квантизация	Замена координаты одним битом	Очень малый размер кода, быстрые побитовые операции	Сильная потеря информации, возможное снижение полноты поиска
Квантизация с кодовыми книгами	Разбиение вектора на блоки и замена каждого блока номером элемента кодовой книги	Хороший компромисс между качеством и памятью, быстрое сравнение через таблицы	Требуется обучение кодовых книг, качество зависит от разбиения на блоки
RaBitQ	Случайное ортогональное преобразование и низкобитное кодирование координат	Не требует обучения кодовых книг, хорошо подходит для высокой размерности	Требуется преобразование запроса, возможны дополнительные скалярные поправки
TurboQuant	Случайное ортогональное преобразование и неравномерные уровни квантизации	Уровни лучше согласованы с типичными значениями координат	Требуется предварительное вычисление уровней и преобразование запроса

2.2 Адаптация алгоритмов квантизации для вычисления приближенной близости в графовом поиске

Для применения квантизации в графовом индексе необходимо определить не только способ хранения вектора, но и способ вычисления близости между запросом и вершиной графа. В HNSW эта операция выполняется многократно при обходе графа. Поэтому она должна быть достаточно быстрой и не должна сильно искажать порядок близости кандидатов.

Пусть каждой вершине v графа соответствует документный вектор x_v . После квантиза-

ции вместо исходного вектора хранится код

$$c_v = E(x_v).$$

При обработке запроса q выполняется подготовка запроса, зависящая от метода квантизации. Затем при посещении вершины v вместо точной близости

$$s(q, x_v) = \langle q, x_v \rangle$$

используется приближенная оценка

$$\widehat{s}(q, c_v).$$

Если векторы нормированы, то поиск по максимальному скалярному произведению эквивалентен поиску по минимальному евклидову расстоянию:

$$\|q - x\|_2^2 = 2 - 2\langle q, x \rangle.$$

Поэтому далее в качестве основной меры близости используется скалярное произведение или его приближение.

Общий порядок использования квантизации в графовом поиске имеет следующий вид:

1. настроить параметры квантизатора, если метод требует обучения;
2. закодировать документные векторы и сохранить их коды;
3. для входящего запроса выполнить подготовку, зависящую от метода;
4. при обходе графа вычислять приближенную близость между запросом и кодом вершины;
5. использовать полученные значения для выбора кандидатов и формирования итогового ответа.

Для разных методов отличаются только три операции: код документа, подготовка запроса и формула приближенной близости. Эти различия приведены в таблице 2.2.

Такая схема позволяет подключать разные методы квантизации к одному и тому же графовому поиску. Логика HNSW при этом не меняется: алгоритм по-прежнему обходит граф, поддерживает множество кандидатов и выбирает ближайшие вершины. Меняется только способ вычисления близости между запросом и вершиной.

Таблица 2.2 – Адаптация методов квантизации для вычисления близости

Метод	Код документа	Подготовка запроса	Приближенная близость
Скалярная квантизация	Целочисленные коды координат	Запрос остается в вещественном виде	$\sum_j q_j(a_j + \Delta_j c_j(x))$
Бинарная квантизация	Битовый код координат или проекций	Запрос также переводится в битовый код	$-H(b(q), b(x))$
Квантизация с кодовыми книгами	Номера элементов кодовых книг для блоков вектора	Строятся таблицы $T_j(t)$ для блоков запроса	$\sum_j T_j(c_j(x))$
RaBitQ	Низкобитные коды координат после случайного поворота	Вычисляется $z = Pq$	$r_x \sum_i z_i \hat{y}_i$, возможно с поправками
TurboQuant	Низкобитные коды координат по неравномерным уровням	Вычисляется $z = Pq$	$r_x \sum_i z_i \hat{y}_i$

Важной особенностью является то, что ошибка квантизации влияет не только на итоговую выдачу, но и на сам маршрут обхода графа. Если приближенная оценка неверно упорядочит соседей текущей вершины, поиск может перейти в менее подходящую область графа. Поэтому для графового поиска важно не только уменьшить среднюю ошибку восстановления вектора, но и сохранить порядок близости между кандидатами.

2.3 Оценка ожидаемого влияния квантизации на качество поиска, скорость обработки запросов и объем памяти индекса

Квантизация влияет на графовый поиск по трем основным направлениям:

- уменьшает объем памяти, необходимый для хранения векторных представлений;
- меняет стоимость вычисления близости между запросом и вершиной графа;
- вносит ошибку, которая может снижать полноту поиска.

Если исходный вектор хранится в формате float32, то на один вектор требуется

$$32d$$

бит. При покоординатной квантизации с r битами на координату требуется

$$rd$$

бит. Следовательно, теоретическая степень сжатия относительно float32 равна

$$\frac{32}{r}.$$

Для бинарной квантизации $r = 1$, поэтому достигается сжатие примерно в 32 раза.

Для квантизации с кодовыми книгами размер кода равен

$$m \log_2 K$$

бит, где m — число блоков, а K — размер кодовой книги. Если $K = 256$, то каждый блок занимает один байт.

Полный размер графового индекса складывается из размера кодов, размера графа и служебных параметров:

$$S_{\text{index}} = S_{\text{graph}} + S_{\text{code}} + S_{\text{params}}.$$

Поэтому фактическое уменьшение полного индекса может быть меньше, чем теоретическое сжатие самих векторов. Это особенно заметно на данных малой размерности, где значимую часть памяти может занимать структура графа.

С точки зрения скорости поиска важна стоимость одного сравнения запроса с документным вектором. Для исходных векторов эта стоимость равна $O(d)$. Скалярная квантизация также требует обработки d координат, но уменьшает объем чтения из памяти. Бинарная квантизация позволяет сравнивать сразу много координат одной побитовой операцией. Квантизация с кодовыми книгами после подготовки таблиц требует только $O(m)$ операций на документ. Методы RaBitQ и TurboQuant требуют обработки d низкобитных координат, а также предварительного преобразования запроса.

Сводная оценка ожидаемого влияния методов приведена в таблице 2.3.

Из этой оценки следует, что универсально лучшего метода квантизации нет. Скалярная квантизация является надежным базовым решением, но дает ограниченное сжатие. Бинарная квантизация обеспечивает минимальный размер кода, но может сильно ухудшить полноту поиска. Квантизация с кодовыми книгами дает хороший практический компромисс, но требует обучения кодовых книг. RaBitQ и TurboQuant наиболее интересны для высокоразмерных векторов, где низкобитное кодирование после случайного ортогонального преобразования может сохранять достаточно информации о направлении вектора.

2.4 Выводы

В данной главе были рассмотрены методы квантизации векторных представлений, которые далее используются в программной реализации и экспериментальном сравнении. По результатам главы можно сделать следующие выводы.

1. Сформулирована задача квантизации применительно к приближенному поиску бли-

Таблица 2.3 – Ожидаемое влияние квантизации на графовый поиск

Метод	Экономия памяти	Ожидаемая скорость	Ожидаемое влияние на полноту
Скалярная квантизация	Умеренная	Близка к float32 или немного выше	Небольшое снижение при 8 битах на координату
Бинарная квантизация	Очень высокая	Высокая за счет побитовых операций	Возможна значительная потеря качества, особенно на малой размерности
Квантизация с кодовыми книгами	Высокая	Высокая после подготовки таблиц	Обычно хороший компромисс между качеством и памятью
RaBitQ	Высокая	Зависит от числа бит и стоимости преобразования запроса	Ожидаемо устойчив на высокоразмерных данных
TurboQuant	Высокая	Сопоставима с RaBitQ	Ожидаемо устойчив на высокоразмерных данных за счет более подходящих уровней квантизации

жайших соседей. Показано, что для поиска важно не только восстановление вектора, но и сохранение порядка близости кандидатов к запросу.

2. Рассмотрены пять методов квантизации, используемых в работе: скалярная квантизация, бинарная квантизация, квантизация с кодовыми книгами, RaBitQ и TurboQuant.
3. Для каждого метода описаны операции кодирования документного вектора, подготовки запроса и вычисления приближенной близости. Это позволяет использовать разные методы квантизации внутри одной схемы графового поиска.
4. Выполнена оценка влияния квантизации на память, скорость и полноту поиска. Наибольший выигрыш по памяти ожидается при сильном сжатии, однако слишком грубая квантизация может снижать полноту, особенно на данных малой размерности.
5. Для дальнейшего экспериментального сравнения выбраны методы, представляющие разные режимы сжатия: надежная базовая скалярная квантизация, предельно компактная бинарная квантизация, квантизация с кодовыми книгами и современные низкобитные методы RaBitQ и TurboQuant.

3. Программная реализация и экспериментальное исследование алгоритмов квантизации для графового приближенного поиска

***Аннотация.** В данной главе описывается программная реализация методов квантизации векторных представлений и их интеграция с графовым индексом HNSW. Рассматриваются реализованные базовые методы, квантизация с кодовыми книгами, а также низкобитные методы RaBitQ и TurboQuant. Далее описываются средства измерения полноты поиска, скорости обработки запросов и размера индекса, приводится экспериментальное сравнение методов на наборах данных различной размерности.*

В предыдущей главе были рассмотрены методы квантизации, выбранные для дальнейшей реализации и сравнения: скалярная квантизация, бинарная квантизация, квантизация с кодовыми книгами, RaBitQ и TurboQuant. В данной главе описывается, как эти методы были реализованы и подключены к графовому индексу HNSW.

Основная цель реализации состояла в том, чтобы сравнить разные способы сжатия векторных представлений в одинаковых условиях. Для этого все методы были приведены к единому порядку работы: настройка параметров, кодирование документных векторов, подготовка запроса и вычисление приближенной близости между запросом и сжатым представлением документа.

3.1 Общая организация программной реализации

Каждый реализованный метод квантизации поддерживает один и тот же набор операций:

- настройка параметров метода, если она необходима;
- кодирование документного вектора;
- подготовка запроса;
- вычисление приближенной близости между запросом и кодом документа.

Такой порядок позволяет использовать разные методы квантизации без изменения основной логики графового поиска. Графовый индекс HNSW при обработке запроса выполняет обход графа, а для сравнения запроса с очередной вершиной вызывает операцию вычисления близости, соответствующую выбранному методу квантизации.

Для вершины v графа вместо исходного документного вектора x_v хранится его сжатый код:

$$c_v = E(x_v).$$

При обработке запроса q точное скалярное произведение

$$\langle q, x_v \rangle$$

заменяется приближенной оценкой

$$\hat{s}(q, c_v).$$

Эта оценка используется при жадном спуске по верхним уровням HNSW, при поддержании множества кандидатов на нижнем уровне и при формировании итогового ответа.

Сравнение реализованных методов по основным операциям приведено в таблице 3.1.

Таблица 3.1 – Основные операции реализованных методов квантизации

Метод	Код документа	Подготовка запроса	Вычисление близости
Скалярная квантизация	Целочисленные коды координат	Запрос остается в вещественном виде	По восстановленным значениям координат
Бинарная квантизация	Упакованный битовый код	Запрос переводится в битовый код	Через расстояние Хэмминга
Product Quantization	Номера элементов кодовых книг	Строятся таблицы близости	Суммированием значений из таблиц
RaBitQ	Низкобитные коды координат после поворота	К запросу применяется тот же поворот	По низкобитным координатам и нормировкам
TurboQuant	Низкобитные коды по неравномерным уровням	К запросу применяется тот же поворот	По восстановленным уровням квантизации

3.2 Реализация базовых методов квантизации векторных представлений

В качестве базовых методов были реализованы скалярная и бинарная квантизация. Они используются как контрольные способы сжатия: скалярная квантизация представляет надежный умеренный вариант, а бинарная квантизация показывает предельный случай сильного сжатия.

3.2.1 Скалярная квантизация

Скалярная квантизация реализована как независимое покоординатное кодирование. Для каждой координаты выбираются параметры диапазона и шаг квантизации. После этого вещественное значение координаты заменяется целочисленным кодом.

В реализации используется восьмибитное представление координаты. Такой режим уменьшает объем хранения векторных представлений примерно в четыре раза по сравнению с хранением координат в виде 32-разрядных вещественных чисел. При этом качество поиска обычно остается близким к поиску без квантизации.

При обработке запроса полное восстановление всех документных векторов не выполняется. Приближенная близость вычисляется непосредственно по целочисленным кодам и параметрам квантизации. Это позволяет не выделять дополнительную память на восстановленные векторы и использовать сжатое представление во время обхода графа.

3.2.2 Бинарная квантизация

Бинарная квантизация реализована как замена координат одним битом. В базовом варианте используется знак координаты. Если координата неотрицательна, в код записывается единица, иначе — ноль.

Для уменьшения объема памяти бинарные коды упаковываются в машинные слова. Сравнение запроса и документа выполняется через расстояние Хэмминга. Для этого используется побитовое исключающее или и операция подсчета единичных битов:

$$H(b(q), b(x)) = \text{popcount}(b(q) \oplus b(x)).$$

Чем меньше расстояние Хэмминга, тем более близкими считаются векторы.

Главное преимущество бинарной квантизации состоит в малом размере кода и быстром вычислении расстояния. Основное ограничение состоит в значительной потере информации, поскольку величины координат не сохраняются. Поэтому этот метод рассматривается как наиболее компактный, но потенциально менее точный вариант.

3.3 Реализация квантизации с кодовыми книгами и низкобитных методов

Помимо базовых методов были реализованы более сложные методы квантизации: Product Quantization, RaBitQ и TurboQuant. Они предназначены для получения более выгодного соотношения между качеством поиска и объемом памяти.

3.3.1 Квантизация с кодовыми книгами

Квантизация с кодовыми книгами реализована на основе метода Product Quantization [3]. Исходный вектор разбивается на несколько блоков, после чего для каждого блока обучается отдельная кодовая книга.

Обучение выполняется по выборке документных векторов. Для каждого блока собираются соответствующие части обучающих векторов, после чего строится кодовая книга из K эталонных блоков. В экспериментах используется значение $K = 256$, так как в этом случае номер элемента кодовой книги можно хранить в одном байте.

Кодирование документного вектора состоит в выборе ближайшего элемента кодовой книги для каждого блока. В результате вместо исходного вектора хранится набор номеров:

$$c(x) = (c_1(x), c_2(x), \dots, c_m(x)).$$

При обработке запроса заранее строятся таблицы близости между блоками запроса и элементами кодовых книг. После этого близость между запросом и документом вычисляется суммированием значений из этих таблиц. Это уменьшает стоимость сравнения, так как она зависит от числа блоков, а не от полной размерности исходного вектора.

3.3.2 RaBitQ и TurboQuant

Методы RaBitQ и TurboQuant реализованы как низкобитная квантизация после случайного ортогонального преобразования [5, 6]. Для документного вектора сначала вычисляется норма, затем применяется ортогональное преобразование, а полученный нормированный вектор кодируется малым числом бит на координату.

В RaBitQ используется знаковое или равномерное низкобитное кодирование координат. При кодировании сохраняются низкобитные значения, норма исходного вектора и дополнительные скалярные параметры, необходимые для более точной оценки близости.

TurboQuant использует ту же общую схему, но отличается выбором уровней квантизации. В отличие от равномерной сетки, уровни TurboQuant заранее согласованы с распределением координат после случайного ортогонального преобразования. Это позволяет точнее кодировать наиболее вероятные значения координат.

Для обоих методов подготовка запроса включает применение того же ортогонального преобразования, которое использовалось при кодировании документных векторов. После этого приближенная близость вычисляется по низкобитным кодам и сохраненным норми-

ровочным коэффициентам.

Преимущество RaBitQ и TurboQuant состоит в том, что им не требуется обучение кодовых книг на конкретном наборе данных. Поэтому они удобны для сравнения с Product Quantization, где этап обучения является обязательным.

3.4 Интеграция реализованных методов с графовым индексом HNSW

Интеграция методов квантизации с HNSW выполняется на уровне операции вычисления близости между запросом и вершиной графа. Логика обхода графа при этом остается неизменной: алгоритм поддерживает очередь кандидатов, просматривает соседей текущих вершин и выбирает вершины с наибольшей близостью к запросу.

В обычном варианте HNSW при посещении вершины v вычисляет точное значение близости:

$$s(q, x_v) = \langle q, x_v \rangle.$$

После подключения квантизации вместо этого используется приближенная оценка:

$$\hat{s}(q, c_v).$$

Общий порядок поиска с использованием квантизации имеет следующий вид:

1. получить запросный вектор q ;
2. выполнить подготовку запроса для выбранного метода квантизации;
3. начать поиск в графе HNSW с входной вершины;
4. при каждом сравнении запроса с вершиной использовать приближенную оценку близости;
5. завершить обход графа по стандартному правилу HNSW;
6. вернуть найденные ближайшие вершины.

Такой способ интеграции позволяет сравнивать методы квантизации при одинаковых параметрах графового поиска. В экспериментах изменяется только способ хранения векторных представлений и способ вычисления близости, а параметры HNSW фиксируются для всех сравниваемых методов.

3.5 Реализация средств измерения

Для экспериментального сравнения были реализованы средства измерения трех основных характеристик:

- полноты поиска;
- числа обработанных запросов в секунду;
- размера построенного индекса.

Полнота поиска вычисляется относительно эталонных ответов точного поиска. Для каждого запроса заранее определяется множество истинных 100 ближайших соседей. После выполнения графового поиска результат сравнивается с этим множеством:

$$R_{100} = \frac{1}{|Q|} \sum_{q \in Q} \frac{|N_{100}(q) \cap \hat{N}_{100}(q)|}{100}.$$

Здесь Q — множество запросов, $N_{100}(q)$ — точные ближайшие соседи, а $\hat{N}_{100}(q)$ — соседи, найденные HNSW с выбранным методом квантизации.

Скорость поиска измеряется как число запросов, обработанных за одну секунду. Время подготовки запроса учитывается как часть общего времени обработки, поскольку в реальной системе эта операция выполняется для каждого входящего запроса.

Размер индекса измеряется с учетом структуры графа HNSW, кодов документных векторов, параметров квантизатора, кодовых книг и дополнительных служебных данных. Это позволяет сравнивать не только теоретический размер кода, но и полный размер индекса, необходимый для выполнения поиска.

3.6 Подготовка наборов данных для экспериментального сравнения

Для экспериментального сравнения были выбраны наборы данных различной размерности. Такой выбор необходим, поскольку влияние квантизации зависит от размерности векторных представлений: на малой размерности сильное сжатие может заметно ухудшать порядок близости, а на высокой размерности низкобитные методы часто работают устойчивее.

Характеристики наборов данных приведены в таблице 3.2.

Перед построением индекса все векторы нормировались по евклидовой норме. Это позволяет использовать скалярное произведение как основную меру близости, поскольку для нормированных векторов максимизация скалярного произведения эквивалентна минимизации евклидова расстояния.

Таблица 3.2 – Характеристики наборов данных

Набор данных	Тип векторных представлений	Число объектов	Размерность	Назначение
GloVe-100	Представления слов [20]	$1,2 \cdot 10^6$	100	Низкая размерность
Deep Image-96	Представления изображений [21]	10^7	96	Большой набор низкоразмерных векторов
DBpedia-1536	Текстовые представления высокой размерности [22]	10^6	1536	Высокая размерность

3.7 Проведение экспериментов

Эксперименты проводились с использованием графового индекса HNSW [4]. Для всех методов квантизации использовались одинаковые параметры графового поиска. Это необходимо, чтобы различия в результатах объяснялись выбранным методом квантизации, а не настройками индекса.

Параметры HNSW приведены в таблице 3.3.

Таблица 3.3 – Параметры HNSW в экспериментах

Параметр	Значение	Назначение
M	32	Максимальное число соседей вершины
$efConstruction$	300	Размер множества кандидатов при построении графа
ef при поиске	100, 500, 1500	Управление соотношением полноты и скорости поиска
Число возвращаемых объектов	100	Размер ответа для вычисления полноты поиска

В качестве базового уровня использовался поиск без квантизации. Далее сравнивались следующие методы:

- скалярная квантизация в 8 бит на координату;
- бинарная квантизация;
- Product Quantization с различной степенью сжатия;
- RaBitQ с различным числом бит на координату;
- TurboQuant с различным числом бит на координату.

Для каждой конфигурации фиксировались полнота поиска, число обработанных запросов в секунду и размер индекса. Далее в таблицах приведены результаты для значения $ef = 1500$, соответствующего режиму высокого качества поиска.

3.8 Сравнение и анализ результатов

3.8.1 Набор данных GloVe-100

Результаты на наборе данных GloVe-100 приведены в таблице 3.4. Набор имеет размерность 100, поэтому значительную часть полного индекса занимает не только хранение векторов, но и структура графа HNSW.

Таблица 3.4 – Результаты на GloVe-100 при $ef = 1500$

Метод	Полнота R_{100}	Запросов в секунду	Размер индекса
Без квантизации	0,9509	96	611 Мбайт
Скалярная квантизация, 8 бит	0,9505	93	273 Мбайт
Product Quantization, 4 бита на координату	0,9501	128	216 Мбайт
Product Quantization, 2 бита на координату	0,9383	169	188 Мбайт
RaBitQ, 4 бита на координату	0,9502	123	233 Мбайт
RaBitQ, 2 бита на координату	0,9360	135	205 Мбайт
TurboQuant, 4 бита на координату	0,9498	120	224 Мбайт
TurboQuant, 2 бита на координату	0,9230	161	196 Мбайт
Бинарная квантизация	0,5266	227	178 Мбайт

На GloVe-100 методы с 4 битами на координату сохраняют полноту почти на уровне поиска без квантизации. При переходе к 2 битам качество начинает снижаться. Бинарная квантизация дает наибольшую скорость и минимальный размер, но полнота поиска оказывается слишком низкой.

3.8.2 Набор данных Deep Image-96

Результаты на наборе Deep Image-96 приведены в таблице 3.5. Этот набор также имеет низкую размерность, но содержит больше объектов, поэтому абсолютный размер индекса существенно выше.

На Deep Image-96 умеренная квантизация также сохраняет высокое качество. При 4 битах на координату Product Quantization, RaBitQ и TurboQuant остаются близки к поиску без квантизации. При 2 битах на координату устойчивее всего показывает себя RaBitQ. Бинарная квантизация снова приводит к значительной потере полноты.

Таблица 3.5 – Результаты на Deep Image-96 при $ef = 1500$

Метод	Полнота R_{100}	Запросов в секунду	Размер индекса
Без квантизации	0,9980	141	4,89 Гбайт
Скалярная квантизация, 8 бит	0,9980	96	2,21 Гбайт
Product Quantization, 4 бита на координату	0,9964	112	1,76 Гбайт
Product Quantization, 2 бита на координату	0,9570	120	1,54 Гбайт
RaBitQ, 4 бита на координату	0,9979	157	1,90 Гбайт
RaBitQ, 2 бита на координату	0,9901	155	1,68 Гбайт
TurboQuant, 4 бита на координату	0,9940	113	1,83 Гбайт
TurboQuant, 2 бита на координату	0,9185	153	1,61 Гбайт
Бинарная квантизация	0,6006	165	1,46 Гбайт

3.8.3 Набор данных DBpedia-1536

Результаты на наборе DBpedia-1536 приведены в таблице 3.6. Этот набор имеет размерность 1536, поэтому квантизация должна давать наиболее заметный выигрыш по памяти.

На DBpedia-1536 квантизация дает наиболее сильный эффект. Поиск без квантизации требует 5,80 Гбайт памяти и обрабатывает 16 запросов в секунду. Низкобитные методы уменьшают размер индекса в несколько раз при сохранении полноты почти на том же уровне.

Product Quantization, RaBitQ и TurboQuant при 2 битах на координату сохраняют полноту 0,9990, но уменьшают размер индекса примерно до 500 Мбайт. Однобитные варианты Product Quantization и RaBitQ также показывают высокую полноту. Это подтверждает, что на высокоразмерных векторах низкобитное представление может сохранять достаточно информации для графового поиска.

3.8.4 Обобщение результатов

По результатам экспериментов можно выделить следующие закономерности.

Во-первых, эффективность квантизации существенно зависит от размерности векторов. На низкоразмерных наборах GloVe-100 и Deep Image-96 сильное сжатие приводит к заметной потере полноты. На высокоразмерном наборе DBpedia-1536 низкобитные методы работают значительно устойчивее.

Во-вторых, скалярная квантизация в 8 бит на координату является надежным базовым

Таблица 3.6 – Результаты на DBpedia-1536 при $ef = 1500$

Метод	Полнота R_{100}	Запросов в секунду	Размер индекса
Без квантизации	0,9990	16	5,80 Гбайт
Скалярная квантизация, 8 бит	0,9990	19	1,55 Гбайт
Product Quantization, 4 бита на координату	0,9990	25	859 Мбайт
Product Quantization, 2 бита на координату	0,9990	38	496 Мбайт
Product Quantization, 1 бит на координату	0,9989	57	315 Мбайт
RaBitQ, 4 бита на координату	0,9990	44	873 Мбайт
RaBitQ, 2 бита на координату	0,9990	54	510 Мбайт
RaBitQ, 1 бит на координату	0,9989	77	329 Мбайт
TurboQuant, 4 бита на координату	0,9991	37	865 Мбайт
TurboQuant, 2 бита на координату	0,9990	54	503 Мбайт
Бинарная квантизация	0,9979	155	318 Мбайт

методом. Она почти не снижает полноту, но уступает низкобитным методам по степени сжатия.

В-третьих, бинарная квантизация обеспечивает максимальную экономию памяти и высокую скорость, но часто слишком сильно ухудшает качество поиска. Поэтому она не является универсальной заменой более точным методам.

В-четвертых, Product Quantization, RaBitQ и TurboQuant показывают наиболее выгодное соотношение между полнотой поиска, скоростью и размером индекса. Особенно хорошо эти методы проявляют себя на данных высокой размерности.

Сводное качественное сравнение приведено в таблице 3.7.

3.9 Выводы

В данной главе была описана реализация методов квантизации, их интеграция с графовым индексом HNSW и экспериментальное сравнение на наборах данных различной размерности.

1. Реализованы скалярная и бинарная квантизация, Product Quantization, RaBitQ и TurboQuant. Для каждого метода реализованы кодирование документных векторов, подготовка запроса и вычисление приближенной близости.

Таблица 3.7 – Сводное сравнение методов квантизации

Метод	Экономия памяти	Влияние на полноту	Итоговая оценка
Скалярная квантизация	Умеренная	Небольшое	Надежный базовый метод, но не лучший по степени сжатия
Бинарная квантизация	Очень высокая	Существенное на малой размерности	Подходит для грубого отбора, но может сильно снижать качество
Product Quantization	Высокая	Небольшое или умеренное	Хороший компромисс между качеством, скоростью и памятью
RaBitQ	Высокая	Небольшое на высокой размерности	Перспективен для высокоразмерных векторных представлений
TurboQuant	Высокая	Небольшое на высокой размерности	Перспективен при ограничениях по памяти и высокой размерности

2. Реализованные методы интегрированы с HNSW через единую операцию вычисления близости между запросом и вершиной графа. Это позволило сравнивать методы при одинаковых параметрах графового поиска.
3. Разработаны средства измерения полноты поиска, числа обработанных запросов в секунду и размера индекса.
4. Эксперименты показали, что на низкоразмерных данных сильное сжатие может заметно снижать полноту поиска, тогда как на высокоразмерных данных низкобитные методы сохраняют высокое качество.
5. Наиболее выгодное соотношение качества, скорости и памяти показали Product Quantization, RaBitQ и TurboQuant. Скалярная квантизация является надежным базовым методом, а бинарная квантизация обеспечивает максимальное сжатие ценой заметной потери качества.

Заключение

В рамках данной работы разработаны и исследованы алгоритмы квантизации векторных представлений для графовых методов приближенного поиска ближайших соседей. Основное внимание было уделено применению квантизации в составе графового индекса HNSW и оценке ее влияния на полноту поиска, скорость обработки запросов и объем памяти индекса.

В соответствии с поставленной целью были получены следующие результаты:

1. Проведен анализ задачи приближенного поиска ближайших соседей и графовых методов поиска. Обоснован выбор HNSW как основного графового метода для исследования влияния квантизации.
2. Изучены и сопоставлены методы квантизации, используемые в работе: скалярная и бинарная квантизация, Product Quantization, RaBitQ и TurboQuant. Для каждого метода рассмотрены способ кодирования векторов, подготовка запроса и вычисление приближенной близости.
3. Реализованы выбранные методы квантизации и выполнена их интеграция с графовым индексом HNSW. При поиске точное вычисление близости между запросом и документным вектором заменяется вычислением приближенной близости по сжатому представлению.
4. Разработаны средства для экспериментального сравнения методов: измерение полноты поиска, числа обработанных запросов в секунду и размера индекса. Это позволило сравнивать методы квантизации при одинаковых параметрах графового поиска.
5. Проведены эксперименты на наборах данных различной размерности. Результаты показали, что на низкоразмерных данных сильное сжатие может заметно снижать полноту поиска, тогда как на высокоразмерных данных низкобитные методы позволяют существенно уменьшить размер индекса при сохранении высокого качества.

Поставленная цель работы достигнута. Наиболее выгодное соотношение между полнотой поиска, скоростью и объемом памяти в проведенных экспериментах показали Product Quantization, RaBitQ и TurboQuant. Скалярная квантизация показала себя как надежный ба-

зовый метод умеренного сжатия, а бинарная квантизация — как наиболее компактный, но менее точный вариант.

Практическая значимость работы состоит в создании программной основы для сравнения различных методов квантизации в составе графового приближенного поиска. Такая реализация может использоваться при выборе способа хранения векторных представлений в системах, где одновременно важны качество поиска, скорость обработки запросов и экономия оперативной памяти.

Перспективы дальнейшей работы включают:

1. исследование двухэтапных схем, в которых квантизация используется для быстрого отбора кандидатов, а более точные представления применяются для итогового упорядочивания;
2. проведение дополнительных экспериментов на более крупных и прикладных наборах векторных представлений;
3. разработку методов автоматического выбора параметров квантизации в зависимости от размерности данных и ограничений по памяти.

Результаты работы создают основу для дальнейшего исследования методов сжатия векторных представлений и их применения в графовых индексах приближенного поиска ближайших соседей.

Список литературы

1. *Indyk P., Motwani R.* Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality // Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing. — ACM, 1998. — С. 604—613. — DOI: 10.1145/276698.276876.
2. *Charikar M. S.* Similarity Estimation Techniques from Rounding Algorithms // Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing. — ACM, 2002. — С. 380—388. — DOI: 10.1145/509907.509965.
3. *Jégou H., Douze M., Schmid C.* Product Quantization for Nearest Neighbor Search // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2011. — Т. 33, № 1. — С. 117—128. — DOI: 10.1109/TPAMI.2010.57.
4. *Malkov Y. A., Yashunin D. A.* Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Т. 42, № 4. — С. 824—836. — DOI: 10.1109/TPAMI.2018.2889473.
5. *Gao J., Long C.* RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search // Proceedings of the ACM on Management of Data. — 2024. — Т. 2, № 3. — 167:1—167:27. — DOI: 10.1145/3654970.
6. TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate / A. Zandieh [и др.] // Computing Research Repository. — 2025. — Т. abs/2504.19874. — DOI: 10.48550/arXiv.2504.19874. — URL: <https://arxiv.org/abs/2504.19874>; arXiv:2504.19874.
7. An Optimal Algorithm for Approximate Nearest Neighbor Searching in Fixed Dimensions / S. Arya [и др.] // Journal of the ACM. — 1998. — Т. 45, № 6. — С. 891—923. — DOI: 10.1145/293347.293348.
8. *Bentley J. L.* Multidimensional Binary Search Trees Used for Associative Searching // Communications of the ACM. — 1975. — Т. 18, № 9. — С. 509—517. — DOI: 10.1145/361002.361007.

9. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions / M. Datar [и др.] // Proceedings of the Twentieth Annual Symposium on Computational Geometry. — ACM, 2004. — С. 253—262. — DOI: 10.1145/997817.997857.
10. Muja M., Lowe D. G. Scalable Nearest Neighbor Algorithms for High Dimensional Data // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2014. — Т. 36, № 11. — С. 2227—2240. — DOI: 10.1109/TPAMI.2014.2321376.
11. Kleinberg J. M. Navigation in a Small World // Nature. — 2000. — Т. 406, № 6798. — С. 845. — DOI: 10.1038/35022643.
12. Pugh W. Skip Lists: A Probabilistic Alternative to Balanced Trees // Communications of the ACM. — 1990. — Т. 33, № 6. — С. 668—676. — DOI: 10.1145/78973.78977.
13. Approximate Nearest Neighbor Algorithm Based on Navigable Small World Graphs / Y. A. Malkov [и др.] // Information Systems. — 2014. — Т. 45. — С. 61—68. — DOI: 10.1016/j.is.2013.10.006.
14. Lloyd S. P. Least Squares Quantization in PCM // IEEE Transactions on Information Theory. — 1982. — Т. 28, № 2. — С. 129—137. — DOI: 10.1109/TIT.1982.1056489.
15. Gersho A., Gray R. M. Vector Quantization and Signal Compression. — Boston : Kluwer Academic Publishers, 1992.
16. Gray R. M., Neuhoff D. L. Quantization // IEEE Transactions on Information Theory. — 1998. — Т. 44, № 6. — С. 2325—2383. — DOI: 10.1109/18.720541.
17. Optimized Product Quantization for Approximate Nearest Neighbor Search / T. Ge [и др.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — IEEE Computer Society, 2013. — С. 2946—2953. — DOI: 10.1109/CVPR.2013.379.
18. André F., Kermarrec A.-M., Le Scouarnec N. Quicker ADC: Unlocking the Hidden Potential of Product Quantization With SIMD // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2021. — Т. 43, № 5. — С. 1666—1677. — DOI: 10.1109/TPAMI.2019.2952606.
19. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search / J. Gao [и др.] // Proceedings of the ACM on Management of Data. — 2025. — Т. 3, № 3. — 202:1—202:26. — DOI: 10.1145/3725413.

20. *Pennington J., Socher R., Manning C. D.* GloVe: Global Vectors for Word Representation // Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. — Association for Computational Linguistics, 2014. — C. 1532—1543. — DOI: 10.3115/v1/D14-1162.
21. *Babenko A., Lempitsky V. S.* Efficient Indexing of Billion-Scale Datasets of Deep Descriptors // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — IEEE Computer Society, 2016. — C. 2055—2063. — DOI: 10.1109/CVPR.2016.226.
22. *Aumüller M., Bernhardsson E., Faithfull A. J.* ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms // Information Systems. — 2020. — T. 87. — C. 101374. — DOI: 10.1016/j.is.2019.02.006.

А. Листинг программной реализации алгоритмов квантизации

В данном приложении приведены основные фрагменты программной реализации алгоритмов квантизации векторных представлений. Код написан на языке C++ и включает общий интерфейс квантизатора, реализации базовых методов квантизации и метода квантизации с кодовыми книгами.

Листинг А.1 – Общий интерфейс квантизатора

```
#pragma once

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <memory>
#include <numeric>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

namespace ann_quantization {

using Vector = std::vector<float>;

struct EncodedVector {
    std::vector<uint8_t> bytes;
    std::vector<uint64_t> words;
    std::vector<float> aux;
};

struct PreparedQuery {
    Vector values;
    std::vector<uint64_t> words;
    std::vector<float> tables;
};

struct QuantizerConfig {
    int dimension = 0;
    int bitsPerDim = 8;
    int subspaces = 16;
    int codebookSize = 256;
    int trainIterations = 20;
    uint32_t seed = 42;
};
```

```

inline float DotProduct(const Vector& a, const Vector& b) {
    float result = 0.0f;
    for (size_t i = 0; i < a.size(); ++i) {
        result += a[i] * b[i];
    }
    return result;
}

inline float SquaredL2(const float* a, const float* b, int size) {
    float result = 0.0f;
    for (int i = 0; i < size; ++i) {
        const float diff = a[i] - b[i];
        result += diff * diff;
    }
    return result;
}

inline float Norm(const Vector& x) {
    return std::sqrt(std::max(DotProduct(x, x), 1e-30f));
}

inline Vector Normalize(const Vector& x) {
    Vector result = x;
    const float n = Norm(result);
    for (float& value : result) {
        value /= n;
    }
    return result;
}

class IQuantizer {
public:
    virtual ~IQuantizer() = default;

    virtual std::string Name() const = 0;

    virtual void Train(const std::vector<Vector>& sample) = 0;

    virtual EncodedVector Encode(const Vector& vector) const = 0;

    virtual PreparedQuery PrepareQuery(const Vector& query) const = 0;

    virtual float Score(
        const PreparedQuery& query,
        const EncodedVector& code
    ) const = 0;

    virtual size_t CodeSizeBytes() const = 0;
};

} // namespace ann_quantization

```

Листинг A.2 – Реализация скалярной квантизации

```
#pragma once
```

```

#include "quantizer_interface.h"

namespace ann_quantization {

class ScalarQuantizer final : public IQuantizer {
public:
    explicit ScalarQuantizer(QuantizerConfig config)
        : config_(config)
    {
        if (config_.dimension <= 0) {
            throw std::invalid_argument("dimension must be positive");
        }
        if (config_.bitsPerDim != 8) {

throw std::invalid_argument("ScalarQuantizer supports 8 bits");
        }

        min_.assign(config_.dimension, 0.0f);
        max_.assign(config_.dimension, 0.0f);
        scale_.assign(config_.dimension, 1.0f);
    }

    std::string Name() const override {
        return "ScalarQuantizer8";
    }

    void Train(const std::vector<Vector>& sample) override {
        if (sample.empty()) {
            throw std::invalid_argument("empty training sample");
        }

        const int dim = config_.dimension;

std::fill(min_.begin(), min_.end(), std::numeric_limits<float>::max());
std::fill(max_.begin(), max_.end(), std::numeric_limits<float>::lowest());

        for (const Vector& vector : sample) {
            if (static_cast<int>(vector.size()) != dim) {
                throw std::invalid_argument("wrong vector dimension");
            }

            for (int i = 0; i < dim; ++i) {
                min_[i] = std::min(min_[i], vector[i]);
                max_[i] = std::max(max_[i], vector[i]);
            }

            for (int i = 0; i < dim; ++i) {
                const float range = max_[i] - min_[i];
                scale_[i] = range > 1e-12f ? range / 255.0f : 1.0f;
            }
        }

        EncodedVector Encode(const Vector& vector) const override {
            const int dim = config_.dimension;

```

```

    if (static_cast<int>(vector.size()) != dim) {
        throw std::invalid_argument("wrong vector dimension");
    }

    EncodedVector result;
    result.bytes.resize(dim);

    for (int i = 0; i < dim; ++i) {
        const float raw = (vector[i] - min_[i]) / scale_[i];
        int code = static_cast<int>(std::lround(raw));
        code = std::max(0, std::min(255, code));
        result.bytes[i] = static_cast<uint8_t>(code);
    }

    return result;
}

PreparedQuery PreparedQuery(const Vector& query) const override {
    if (static_cast<int>(query.size()) != config_.dimension) {
        throw std::invalid_argument("wrong query dimension");
    }

    PreparedQuery prepared;
    prepared.values = query;
    return prepared;
}

float Score(
    const PreparedQuery& query,
    const EncodedVector& code
) const override {
    const int dim = config_.dimension;

    float result = 0.0f;
    for (int i = 0; i < dim; ++i) {
        const float restored = min_[i] + scale_[i] * code.bytes[i];
        result += query.values[i] * restored;
    }

    return result;
}

size_t CodeSizeBytes() const override {
    return static_cast<size_t>(config_.dimension);
}

private:
    QuantizerConfig config_;
    Vector min_;
    Vector max_;
    Vector scale_;
};

} // namespace ann_quantization

```

Листинг A.3 – Реализация бинарной квантизации

```
#pragma once

#include "quantizer_interface.h"

namespace ann_quantization {

class BinaryQuantizer final : public IQuantizer {
public:
    explicit BinaryQuantizer(QuantizerConfig config)
        : config_(config)
    {
        if (config_.dimension <= 0) {
            throw std::invalid_argument("dimension must be positive");
        }
        wordsCount_ = (config_.dimension + 63) / 64;
    }

    std::string Name() const override {
        return "BinaryQuantizer";
    }

    void Train(const std::vector<Vector>&) override {
        // The basic binary quantizer does not require training.
    }

    EncodedVector Encode(const Vector& vector) const override {
        if (static_cast<int>(vector.size()) != config_.dimension) {
            throw std::invalid_argument("wrong vector dimension");
        }

        EncodedVector result;
        result.words.assign(wordsCount_, 0);

        for (int i = 0; i < config_.dimension; ++i) {
            if (vector[i] >= 0.0f) {
                const int word = i / 64;
                const int bit = i % 64;
                result.words[word] |= (uint64_t{1} << bit);
            }
        }

        return result;
    }

    PreparedQuery PrepareQuery(const Vector& query) const override {
        if (static_cast<int>(query.size()) != config_.dimension) {
            throw std::invalid_argument("wrong query dimension");
        }

        PreparedQuery prepared;
        prepared.words.assign(wordsCount_, 0);

        for (int i = 0; i < config_.dimension; ++i) {
            if (query[i] >= 0.0f) {
                const int word = i / 64;
```

```

        const int bit = i % 64;
        prepared.words[word] |= (uint64_t{1} << bit);
    }
}

return prepared;
}

float Score(
    const PreparedQuery& query,
    const EncodedVector& code
) const override {
    int hamming = 0;

    for (int i = 0; i < wordsCount_; ++i) {
        const uint64_t diff = query.words[i] ^ code.words[i];
        hamming += PopCount(diff);
    }

    return -static_cast<float>(hamming);
}

size_t CodeSizeBytes() const override {
    return static_cast<size_t>(wordsCount_) * sizeof(uint64_t);
}

private:
    static int PopCount(uint64_t value) {
#ifdef __GNUG__ || defined(__clang__)
        return __builtin_popcountll(value);
#else
        int count = 0;
        while (value != 0) {
            value &= value - 1;
            ++count;
        }
        return count;
#endif
    }

private:
    QuantizerConfig config_;
    int wordsCount_ = 0;
};

} // namespace ann_quantization

```

Листинг А.4 – Реализация квантизации с кодовыми книгами

```

#pragma once

#include "quantizer_interface.h"

namespace ann_quantization {

class ProductQuantizer final : public IQuantizer {

```

```

public:
    explicit ProductQuantizer(QuantizerConfig config)
        : config_(config)
    {
        if (config_.dimension <= 0) {
            throw std::invalid_argument("dimension must be positive");
        }
        if (config_.subspaces <= 0) {
            throw std::invalid_argument("subspaces must be positive");
        }
        if (config_.dimension % config_.subspaces != 0) {

throw std::invalid_argument("dimension must be divisible by subspaces");
        }
        if (config_.codebookSize != 256) {

throw std::invalid_argument("only codebook size 256 is used");
        }

        blockDim_ = config_.dimension / config_.subspaces;
        codebooks_.resize(
            static_cast<size_t>(config_.subspaces)
            * config_.codebookSize
            * blockDim_,
            0.0f
        );
    }

    std::string Name() const override {
        return "ProductQuantizer";
    }

    void Train(const std::vector<Vector>& sample) override {
        if (sample.empty()) {
            throw std::invalid_argument("empty training sample");
        }

        for (int part = 0; part < config_.subspaces; ++part) {
            TrainCodebook(sample, part);
        }
    }

    EncodedVector Encode(const Vector& vector) const override {
        if (static_cast<int>(vector.size()) != config_.dimension) {
            throw std::invalid_argument("wrong vector dimension");
        }

        EncodedVector result;
        result.bytes.resize(config_.subspaces);

        for (int part = 0; part < config_.subspaces; ++part) {
            const float* block = vector.data() + part * blockDim_;
            const int nearest = FindNearestCodeword(part, block);
            result.bytes[part] = static_cast<uint8_t>(nearest);
        }

        return result;
    }

```

```

}

PreparedQuery PrepareQuery(const Vector& query) const override {
    if (static_cast<int>(query.size()) != config_.dimension) {
        throw std::invalid_argument("wrong query dimension");
    }

    PreparedQuery prepared;
    prepared.tables.assign(

static_cast<size_t>(config_.subspaces) * config_.codebookSize,
        0.0f
    );

    for (int part = 0; part < config_.subspaces; ++part) {
        const float* queryBlock = query.data() + part * blockDim_;

        for (int code = 0; code < config_.codebookSize; ++code) {
            const float* centroid = Codeword(part, code);
            float dot = 0.0f;

            for (int i = 0; i < blockDim_; ++i) {
                dot += queryBlock[i] * centroid[i];
            }

prepared.tables[part * config_.codebookSize + code] = dot;
        }

        return prepared;
    }

    float Score(
        const PreparedQuery& query,
        const EncodedVector& code
    ) const override {
        float result = 0.0f;

        for (int part = 0; part < config_.subspaces; ++part) {
            const int codeId = static_cast<int>(code.bytes[part]);

result += query.tables[part * config_.codebookSize + codeId];
        }

        return result;
    }

    size_t CodeSizeBytes() const override {
        return static_cast<size_t>(config_.subspaces);
    }

private:
    float* Codeword(int part, int code) {
        const size_t offset =
            (static_cast<size_t>(part) * config_.codebookSize + code)
            * blockDim_;

```



```

        return codebooks_.data() + offset;
    }

    const float* Codeword(int part, int code) const {
        const size_t offset =
            (static_cast<size_t>(part) * config_.codebookSize + code)
            * blockDim_;
        return codebooks_.data() + offset;
    }

    void TrainCodebook(const std::vector<Vector>& sample, int part) {
        std::mt19937 generator(config_.seed + static_cast<uint32_t>(part));
        std::uniform_int_distribution<size_t> distribution(0, sample.size() - 1);

        for (int code = 0; code < config_.codebookSize; ++code) {
            const Vector& source = sample[distribution(generator)];
            const float* block = source.data() + part * blockDim_;
            std::copy(block, block + blockDim_, Codeword(part, code));
        }

        std::vector<int> assignment(sample.size(), 0);
        std::vector<float> sums(
            static_cast<size_t>(config_.codebookSize) * blockDim_,
            0.0f
        );
        std::vector<int> counts(config_.codebookSize, 0);

        for (int iter = 0; iter < config_.trainIterations; ++iter) {
            std::fill(sums.begin(), sums.end(), 0.0f);
            std::fill(counts.begin(), counts.end(), 0);

            for (size_t row = 0; row < sample.size(); ++row) {
                const float* block = sample[row].data() + part * blockDim_;
                const int nearest = FindNearestCodeword(part, block);
                assignment[row] = nearest;
                ++counts[nearest];

                float* sum = sums.data() + nearest * blockDim_;
                for (int i = 0; i < blockDim_; ++i) {
                    sum[i] += block[i];
                }
            }

            for (int code = 0; code < config_.codebookSize; ++code) {
                if (counts[code] == 0) {
                    const Vector& source = sample[distribution(generator)];
                    const float* block = source.data() + part * blockDim_;
                    std::copy(block, block + blockDim_, Codeword(part, code));
                    continue;
                }
            }
        }
    }

```

```

        float* centroid = Codeword(part, code);
        const float* sum = sums.data() + code * blockDim_;

        for (int i = 0; i < blockDim_; ++i) {
centroid[i] = sum[i] / static_cast<float>(counts[code]);
        }
    }
}

int FindNearestCodeword(int part, const float* block) const {
    int bestCode = 0;
    float bestDistance = std::numeric_limits<float>::max();

    for (int code = 0; code < config_.codebookSize; ++code) {
        const float* centroid = Codeword(part, code);

const float distance = SquaredL2(block, centroid, blockDim_);

        if (distance < bestDistance) {
            bestDistance = distance;
            bestCode = code;
        }
    }

    return bestCode;
}

private:
    QuantizerConfig config_;
    int blockDim_ = 0;
    std::vector<float> codebooks_;
};

} // namespace ann_quantization

\begin{lstlisting}[language=C++, captionВспомогательные={ классы для низкобитных}]
#pragma once

#include "quantizer_interface.h"

namespace ann_quantization {

class BitWriter {
public:
    explicit BitWriter(int bitsPerValue)
        : bitsPerValue_(bitsPerValue)
    {
    }

    void Resize(size_t valuesCount) {

const size_t bits = valuesCount * static_cast<size_t>(bitsPerValue_);
        data_.assign((bits + 7) / 8, 0);
    }
}

```

```

    void Set(size_t index, uint32_t value) {

const size_t bitOffset = index * static_cast<size_t>(bitsPerValue_);

        for (int bit = 0; bit < bitsPerValue_; ++bit) {
            if ((value >> bit) & 1U) {
                const size_t position = bitOffset + bit;

data_[position / 8] |= static_cast<uint8_t>(1U << (position % 8));
            }
        }

const std::vector<uint8_t>& Data() const {
    return data_;
}

private:
    int bitsPerValue_;
    std::vector<uint8_t> data_;
};

class BitReader {
public:
    BitReader(const std::vector<uint8_t>& data, int bitsPerValue)
        : data_(data)
        , bitsPerValue_(bitsPerValue)
    {
    }

    uint32_t Get(size_t index) const {

const size_t bitOffset = index * static_cast<size_t>(bitsPerValue_);
    uint32_t result = 0;

        for (int bit = 0; bit < bitsPerValue_; ++bit) {
            const size_t position = bitOffset + bit;
            const uint8_t current =

static_cast<uint8_t>((data_[position / 8] >> (position % 8)) & 1U);
            result |= static_cast<uint32_t>(current) << bit;
        }

        return result;
    }

private:
    const std::vector<uint8_t>& data_;
    int bitsPerValue_;
};

class DenseOrthogonalTransform {
public:
    DenseOrthogonalTransform() = default;

    DenseOrthogonalTransform(int dimension, uint32_t seed)
        : dimension_(dimension)

```

```

{
    Build(dimension, seed);
}

void Build(int dimension, uint32_t seed) {
    dimension_ = dimension;

    matrix_.assign(static_cast<size_t>(dimension_) * dimension_, 0.0f);

    std::mt19937 generator(seed);
    std::normal_distribution<float> normal(0.0f, 1.0f);

    for (float& value : matrix_) {
        value = normal(generator);
    }

    // Modified Gram-Schmidt orthogonalization.
    for (int col = 0; col < dimension_; ++col) {
        for (int prev = 0; prev < col; ++prev) {
            float projection = 0.0f;

            for (int row = 0; row < dimension_; ++row) {
                projection += At(row, col) * At(row, prev);
            }

            for (int row = 0; row < dimension_; ++row) {
                At(row, col) -= projection * At(row, prev);
            }
        }

        float norm = 0.0f;
        for (int row = 0; row < dimension_; ++row) {
            norm += At(row, col) * At(row, col);
        }
        norm = std::sqrt(std::max(norm, 1e-30f));

        for (int row = 0; row < dimension_; ++row) {
            At(row, col) /= norm;
        }
    }
}

Vector Apply(const Vector& x) const {
    if (static_cast<int>(x.size()) != dimension_) {
        throw std::invalid_argument("wrong vector dimension");
    }

    Vector result(dimension_, 0.0f);

    for (int row = 0; row < dimension_; ++row) {
        float value = 0.0f;

        for (int col = 0; col < dimension_; ++col) {
            value += At(row, col) * x[col];
        }

        result[row] = value;
    }
}

```

```

    }

    return result;
}

private:
    float& At(int row, int col) {
        return matrix_[static_cast<size_t>(row) * dimension_ + col];
    }

    const float& At(int row, int col) const {
        return matrix_[static_cast<size_t>(row) * dimension_ + col];
    }

private:
    int dimension_ = 0;
    std::vector<float> matrix_;
};

} // namespace ann_quantization

```

Листинг А.5 – Реализация квантизатора RaBitQ

```

#pragma once

#include "quantizer_interface.h"
#include "lowbit_helpers.h"

namespace ann_quantization {

class RaBitQuantizer final : public IQuantizer {
public:
    explicit RaBitQuantizer(QuantizerConfig config)
        : config_(config)
        , rotation_(config.dimension, config.seed)
    {
        if (config_.dimension <= 0) {
            throw std::invalid_argument("dimension must be positive");
        }
        if (config_.bitsPerDim != 1 &&
            config_.bitsPerDim != 2 &&
            config_.bitsPerDim != 4 &&
            config_.bitsPerDim != 8) {
            throw std::invalid_argument("unsupported bitness");
        }

        levelsCount_ = 1 << config_.bitsPerDim;

        bound_ = 4.0f / std::sqrt(static_cast<float>(config_.dimension));
    }

    std::string Name() const override {
        return "RaBitQ";
    }

    void Train(const std::vector<Vector>&) override {

```

```

        // RaBitQ does not require data-dependent codebook training.
    }

EncodedVector Encode(const Vector& vector) const override {
    if (static_cast<int>(vector.size()) != config_.dimension) {
        throw std::invalid_argument("wrong vector dimension");
    }

    const float norm = Norm(vector);
    Vector rotated = rotation_.Apply(vector);

    for (float& value : rotated) {
        value /= norm;
    }

    EncodedVector result;
    BitWriter writer(config_.bitsPerDim);
    writer.Resize(config_.dimension);

    for (int i = 0; i < config_.dimension; ++i) {
        const uint32_t code = QuantizeUniform(rotated[i]);
        writer.Set(static_cast<size_t>(i), code);
    }

    result.bytes = writer.Data();
    result.aux.push_back(norm);
    return result;
}

PreparedQuery PrepareQuery(const Vector& query) const override {
    PreparedQuery prepared;
    prepared.values = rotation_.Apply(query);
    return prepared;
}

float Score(
    const PreparedQuery& query,
    const EncodedVector& code
) const override {
    const float norm = code.aux.empty() ? 1.0f : code.aux[0];
    BitReader reader(code.bytes, config_.bitsPerDim);

    float result = 0.0f;

    for (int i = 0; i < config_.dimension; ++i) {
        const uint32_t packed = reader.Get(static_cast<size_t>(i));
        const float restored = DequantizeUniform(packed);
        result += query.values[i] * restored;
    }

    return norm * result;
}

size_t CodeSizeBytes() const override {
    const size_t bits =
static_cast<size_t>(config_.dimension) * config_.bitsPerDim;

```

```

        return (bits + 7) / 8 + sizeof(float);
    }

private:
    uint32_t QuantizeUniform(float value) const {
        value = std::max(-bound_, std::min(bound_, value));

        const float normalized = (value + bound_) / (2.0f * bound_);
        int code = static_cast<int>(

std::lround(normalized * static_cast<float>(levelsCount_ - 1))
        );

        code = std::max(0, std::min(levelsCount_ - 1, code));
        return static_cast<uint32_t>(code);
    }

    float DequantizeUniform(uint32_t code) const {
        const float normalized =

static_cast<float>(code) / static_cast<float>(levelsCount_ - 1);
        return -bound_ + 2.0f * bound_ * normalized;
    }

private:
    QuantizerConfig config_;
    DenseOrthogonalTransform rotation_;
    int levelsCount_ = 0;
    float bound_ = 1.0f;
};

} // namespace ann_quantization

```

Листинг А.6 – Реализация квантизатора TurboQuant

```

#pragma once

#include "quantizer_interface.h"
#include "lowbit_helpers.h"

namespace ann_quantization {

class TurboQuantizer final : public IQuantizer {
public:
    explicit TurboQuantizer(QuantizerConfig config)
        : config_(config)
        , rotation_(config.dimension, config.seed)
    {
        if (config_.dimension <= 0) {
            throw std::invalid_argument("dimension must be positive");
        }
        if (config_.bitsPerDim != 2 &&
            config_.bitsPerDim != 4 &&
            config_.bitsPerDim != 8) {
            throw std::invalid_argument("unsupported bitness");
        }
    }

```

```

        levelsCount_ = 1 << config_.bitsPerDim;
        BuildLevels();
    }

    std::string Name() const override {
        return "TurboQuant";
    }

    void Train(const std::vector<Vector>&) override {
        // TurboQuant uses levels that depend on dimension and bitness.
        // No data-dependent training is required.
    }

    EncodedVector Encode(const Vector& vector) const override {
        if (static_cast<int>(vector.size()) != config_.dimension) {
            throw std::invalid_argument("wrong vector dimension");
        }

        const float norm = Norm(vector);
        Vector rotated = rotation_.Apply(vector);

        for (float& value : rotated) {
            value /= norm;
        }

        BitWriter writer(config_.bitsPerDim);
        writer.Resize(config_.dimension);

        for (int i = 0; i < config_.dimension; ++i) {
writer.Set(static_cast<size_t>(i), FindNearestLevel(rotated[i]));
        }

        EncodedVector result;
        result.bytes = writer.Data();
        result.aux.push_back(norm);
        return result;
    }

    PreparedQuery PrepareQuery(const Vector& query) const override {
        PreparedQuery prepared;
        prepared.values = rotation_.Apply(query);
        return prepared;
    }

    float Score(
        const PreparedQuery& query,
        const EncodedVector& code
    ) const override {
        const float norm = code.aux.empty() ? 1.0f : code.aux[0];
        BitReader reader(code.bytes, config_.bitsPerDim);

        float result = 0.0f;

        for (int i = 0; i < config_.dimension; ++i) {
            const uint32_t packed = reader.Get(static_cast<size_t>(i));

```



```

        result += query.values[i] * levels_[packed];
    }

    return norm * result;
}

size_t CodeSizeBytes() const override {
    const size_t bits =

static_cast<size_t>(config_.dimension) * config_.bitsPerDim;
    return (bits + 7) / 8 + sizeof(float);
}

private:
    void BuildLevels() {
        levels_.assign(levelsCount_, 0.0f);

        const float bound =
            4.0f / std::sqrt(static_cast<float>(config_.dimension));

        // The levels are denser near zero, where coordinates after a random
        // orthogonal transform are observed most often.
        for (int i = 0; i < levelsCount_; ++i) {
            const float u =
                -1.0f + 2.0f * (static_cast<float>(i) + 0.5f)
                / static_cast<float>(levelsCount_);

            const float sign = u < 0.0f ? -1.0f : 1.0f;
            const float value =
                sign * std::pow(std::abs(u), 1.7f);

            levels_[i] = bound * value;
        }

        std::sort(levels_.begin(), levels_.end());
    }

    uint32_t FindNearestLevel(float value) const {
        auto it = std::lower_bound(levels_.begin(), levels_.end(), value);

        if (it == levels_.begin()) {
            return 0;
        }
        if (it == levels_.end()) {
            return static_cast<uint32_t>(levels_.size() - 1);
        }

        const size_t right = static_cast<size_t>(it - levels_.begin());
        const size_t left = right - 1;

        const float distLeft = std::abs(value - levels_[left]);
        const float distRight = std::abs(value - levels_[right]);

        return static_cast<uint32_t>(distLeft <= distRight ? left : right);
    }

```

```

    }

private:
    QuantizerConfig config_;
    DenseOrthogonalTransform rotation_;
    int levelsCount_ = 0;
    std::vector<float> levels_;
};

} // namespace ann_quantization

```

Листинг А.7 – Интеграция квантизаторов с графовым индексом HNSW

```

#pragma once

#include "quantizer_interface.h"

#include <queue>
#include <unordered_set>

namespace ann_quantization {

struct SearchResult {
    int id = -1;
    float score = 0.0f;
};

class QuantizedHnswIndex {
public:
    QuantizedHnswIndex(
        std::shared_ptr<IQuantizer> quantizer,
        int efSearch
    )
        : quantizer_(std::move(quantizer))
        , efSearch_(efSearch)
    {
        if (!quantizer_) {
            throw std::invalid_argument("quantizer is null");
        }
    }

    void SetGraph(std::vector<std::vector<int>> graph) {
        graph_ = std::move(graph);
    }

    void SetEntryPoint(int entryPoint) {
        entryPoint_ = entryPoint;
    }

    void AddEncodedVector(const Vector& vector) {
        codes_.push_back(quantizer_->Encode(vector));
    }

    std::vector<SearchResult> Search(
        const Vector& query,
        int topK
    )

```

```

) const {
    if (codes_.empty()) {
        return {};
    }

    if (entryPoint_ < 0 || entryPoint_ >= static_cast<int>(codes_.size())) {
        throw std::runtime_error("invalid entry point");
    }

    PreparedQuery prepared = quantizer_->PrepareQuery(query);

    struct Candidate {
        int id;
        float score;
    };

    struct ByBestScore {

    bool operator()(const Candidate& a, const Candidate& b) const {
        return a.score < b.score;
    }
    };

    struct ByWorstScore {

    bool operator()(const Candidate& a, const Candidate& b) const {
        return a.score > b.score;
    }
    };

    std::priority_queue<Candidate, std::vector<Candidate>, ByBestScore>
        candidates;

    std::priority_queue<Candidate, std::vector<Candidate>, ByWorstScore>
        best;

    std::vector<uint8_t> visited(codes_.size(), 0);

    const float entryScore =
        quantizer_->Score(prepared, codes_[entryPoint_]);

    candidates.push({entryPoint_, entryScore});
    best.push({entryPoint_, entryScore});
    visited[entryPoint_] = 1;

    while (!candidates.empty()) {
        const Candidate current = candidates.top();
        candidates.pop();

        if (static_cast<int>(best.size()) >= efSearch_ &&
            current.score < best.top().score) {
            break;
        }

        for (int neighbor : graph_[current.id]) {

```

```

        if (neighbor < 0 ||
            neighbor >= static_cast<int>(codes_.size()) ||
            visited[neighbor]) {
            continue;
        }

        visited[neighbor] = 1;

        const float score =
            quantizer_>Score(prepared, codes_[neighbor]);

        if (static_cast<int>(best.size()) < efSearch_ ||
            score > best.top().score) {
            candidates.push({neighbor, score});
            best.push({neighbor, score});

            if (static_cast<int>(best.size()) > efSearch_) {
                best.pop();
            }
        }
    }

    std::vector<SearchResult> result;
    result.reserve(best.size());

    while (!best.empty()) {
        result.push_back({best.top().id, best.top().score});
        best.pop();
    }

    std::sort(
        result.begin(),
        result.end(),
        [](const SearchResult& a, const SearchResult& b) {
            return a.score > b.score;
        }
    );

    if (static_cast<int>(result.size()) > topK) {
        result.resize(topK);
    }

    return result;
}

size_t SizeBytes() const {
    size_t result = 0;

    for (const auto& code : codes_) {
        result += code.bytes.size();
        result += code.words.size() * sizeof(uint64_t);
        result += code.aux.size() * sizeof(float);
    }

    for (const auto& neighbors : graph_) {
        result += neighbors.size() * sizeof(int);
    }
}

```

```

    }

    return result;
}

private:
    std::shared_ptr<IQuantizer> quantizer_;
    int efSearch_ = 100;
    int entryPoint_ = 0;

    std::vector<EncodedVector> codes_;
    std::vector<std::vector<int>> graph_;
};

} // namespace ann_quantization

```

Листинг А.8 – Средства измерения полноты поиска и скорости

```

#pragma once

#include "quantizer_interface.h"
#include "hnsf_integration.h"

#include <chrono>
#include <fstream>
#include <iostream>
#include <sstream>

namespace ann_quantization {

struct ExperimentResult {
    std::string dataset;
    std::string method;
    int dimension = 0;
    int efSearch = 0;
    float recallAt100 = 0.0f;
    double queriesPerSecond = 0.0;
    size_t indexSizeBytes = 0;
};

inline float RecallAtK(
    const std::vector<std::vector<int>>& groundTruth,
    const std::vector<std::vector<int>>& predicted,
    int k
) {
    if (groundTruth.size() != predicted.size()) {
        throw std::invalid_argument("different number of queries");
    }

    double total = 0.0;

    for (size_t queryId = 0; queryId < groundTruth.size(); ++queryId) {
        std::unordered_set<int> trueIds;

        for (int i = 0; i < k && i < static_cast<int>(groundTruth[queryId].size());

```

```

        trueIds.insert(groundTruth[queryId][i]);
    }

    int hit = 0;

    for (int i = 0; i < k && i < static_cast<int>(predicted[queryId].size()); ++i) {
        if (trueIds.count(predicted[queryId][i]) != 0) {
            ++hit;
        }
    }

    total += static_cast<double>(hit) / static_cast<double>(k);
}

return static_cast<float>(total / static_cast<double>(groundTruth.size()));
}

inline std::vector<std::vector<int>> RunSearchBatch(
    const QuantizedHnswIndex& index,
    const std::vector<Vector>& queries,
    int topK
) {
    std::vector<std::vector<int>> result;
    result.reserve(queries.size());

    for (const Vector& query : queries) {
        const auto found = index.Search(query, topK);

        std::vector<int> ids;
        ids.reserve(found.size());

        for (const SearchResult& item : found) {
            ids.push_back(item.id);
        }

        result.push_back(std::move(ids));
    }

    return result;
}

inline ExperimentResult RunExperiment(
    const std::string& datasetName,
    const std::vector<Vector>& baseVectors,
    const std::vector<Vector>& trainSample,
    const std::vector<Vector>& queries,
    const std::vector<std::vector<int>>& groundTruth,
    const std::vector<std::vector<int>>& hnswGraph,
    std::shared_ptr<IQuantizer> quantizer,
    int efSearch
) {
    if (baseVectors.empty()) {
        throw std::invalid_argument("empty base vectors");
    }

    quantizer->Train(trainSample);

```

```

QuantizedHnswIndex index(quantizer, efSearch);
index.SetGraph(hnswGraph);
index.SetEntryPoint(0);

for (const Vector& vector : baseVectors) {
    index.AddEncodedVector(vector);
}

const auto start = std::chrono::steady_clock::now();
const std::vector<std::vector<int>> predicted =
    RunSearchBatch(index, queries, 100);
const auto finish = std::chrono::steady_clock::now();

const double seconds =
    std::chrono::duration<double>(finish - start).count();

ExperimentResult result;
result.dataset = datasetName;
result.method = quantizer->Name();
result.dimension = static_cast<int>(baseVectors.front().size());
result.efSearch = efSearch;
result.recallAt100 = RecallAtK(groundTruth, predicted, 100);
result.queriesPerSecond =
    static_cast<double>(queries.size()) / std::max(seconds, 1e-9);
result.indexSizeBytes = index.SizeBytes();

return result;
}

inline void PrintCsvHeader(std::ostream& out) {
    out
        << "dataset,method,dimension,ef,recall_at_100,"
        << "queries_per_second,index_size_bytes\n";
}

inline void PrintCsvRow(std::ostream& out, const ExperimentResult& result) {
    out
        << result.dataset << ", "
        << result.method << ", "
        << result.dimension << ", "
        << result.efSearch << ", "
        << result.recallAt100 << ", "
        << result.queriesPerSecond << ", "
        << result.indexSizeBytes << "\n";
}

} // namespace ann_quantization

```

Листинг А.9 – Пример запуска серии экспериментов

```

#include "quantizer_interface.h"
#include "scalar_quantizer.h"
#include "binary_quantizer.h"
#include "product_quantizer.h"
#include "rabbitq_quantizer.h"

```

```

#include "turboquant_quantizer.h"
#include "metrics_runner.h"

using namespace ann_quantization;

namespace {

std::vector<Vector> LoadVectors(const std::string& path, int dimension) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open vector file: " + path);
    }

    uint64_t count = 0;
    input.read(reinterpret_cast<char*>(&count), sizeof(count));

    std::vector<Vector> vectors(count, Vector(dimension));

    for (uint64_t i = 0; i < count; ++i) {
        input.read(
            reinterpret_cast<char*>(vectors[i].data()),
            sizeof(float) * dimension
        );
        vectors[i] = Normalize(vectors[i]);
    }

    return vectors;
}

std::vector<std::vector<int>> LoadIntLists(const std::string& path) {
    std::ifstream input(path, std::ios::binary);
    if (!input) {
        throw std::runtime_error("cannot open list file: " + path);
    }

    uint64_t rows = 0;
    uint64_t width = 0;

    input.read(reinterpret_cast<char*>(&rows), sizeof(rows));
    input.read(reinterpret_cast<char*>(&width), sizeof(width));

    std::vector<std::vector<int>> result(rows, std::vector<int>(width));

    for (uint64_t i = 0; i < rows; ++i) {
        input.read(
            reinterpret_cast<char*>(result[i].data()),
            sizeof(int) * width
        );
    }

    return result;
}

std::vector<Vector> TakeSample(
    const std::vector<Vector>& vectors,
    size_t sampleSize

```



```

) {
    std::vector<Vector> sample;
    sample.reserve(std::min(sampleSize, vectors.size()));

    for (size_t i = 0; i < vectors.size() && i < sampleSize; ++i) {
        sample.push_back(vectors[i]);
    }

    return sample;
}

std::vector<std::shared_ptr<IQuantizer>> MakeQuantizers(
    int dimension,
    uint32_t seed
) {
    std::vector<std::shared_ptr<IQuantizer>> result;

    QuantizerConfig scalar;
    scalar.dimension = dimension;
    scalar.bitsPerDim = 8;
    scalar.seed = seed;
    result.push_back(std::make_shared<ScalarQuantizer>(scalar));

    QuantizerConfig binary;
    binary.dimension = dimension;
    binary.bitsPerDim = 1;
    binary.seed = seed;
    result.push_back(std::make_shared<BinaryQuantizer>(binary));

    QuantizerConfig pq4;
    pq4.dimension = dimension;
    pq4.subspaces = std::max(1, dimension / 8);
    pq4.codebookSize = 256;
    pq4.trainIterations = 20;
    pq4.seed = seed;
    if (dimension % pq4.subspaces == 0) {
        result.push_back(std::make_shared<ProductQuantizer>(pq4));
    }

    QuantizerConfig pq2;
    pq2.dimension = dimension;
    pq2.subspaces = std::max(1, dimension / 16);
    pq2.codebookSize = 256;
    pq2.trainIterations = 20;
    pq2.seed = seed + 1;
    if (dimension % pq2.subspaces == 0) {
        result.push_back(std::make_shared<ProductQuantizer>(pq2));
    }

    for (int bits : {1, 2, 4}) {
        QuantizerConfig rabbitq;
        rabbitq.dimension = dimension;
        rabbitq.bitsPerDim = bits;
        rabbitq.seed = seed + 10 + bits;
        result.push_back(std::make_shared<RaBitQuantizer>(rabbitq));
    }
}

```

```

    for (int bits : {2, 4}) {
        QuantizerConfig turbo;
        turbo.dimension = dimension;
        turbo.bitsPerDim = bits;
        turbo.seed = seed + 20 + bits;
        result.push_back(std::make_shared<TurboQuantizer>(turbo));
    }

    return result;
}

} // namespace

int main(int argc, char** argv) {
    if (argc < 7) {
        std::cerr

<< "Usage: run_experiments DATASET DIM BASE QUERIES GT GRAPH\n";
        return 1;
    }

    const std::string dataset = argv[1];
    const int dimension = std::stoi(argv[2]);
    const std::string basePath = argv[3];
    const std::string queriesPath = argv[4];
    const std::string groundTruthPath = argv[5];
    const std::string graphPath = argv[6];

    const std::vector<Vector> baseVectors =
        LoadVectors(basePath, dimension);

    const std::vector<Vector> queries =
        LoadVectors(queriesPath, dimension);

    const std::vector<std::vector<int>> groundTruth =
        LoadIntLists(groundTruthPath);

    const std::vector<std::vector<int>> hnswGraph =
        LoadIntLists(graphPath);

    const std::vector<Vector> trainSample =
        TakeSample(baseVectors, 100000);

    const std::vector<int> efValues = {100, 500, 1500};
    const auto quantizers = MakeQuantizers(dimension, 42);

    PrintCsvHeader(std::cout);

    for (int ef : efValues) {
        for (const auto& quantizer : quantizers) {
            const ExperimentResult result =
                RunExperiment(
                    dataset,
                    baseVectors,
                    trainSample,
                    queries,
                    groundTruth,

```

```
        hnswhGraph,  
        quantizer,  
        ef  
    );  
  
    PrintCsvRow(std::cout, result);  
}  
}  
  
return 0;  
}
```